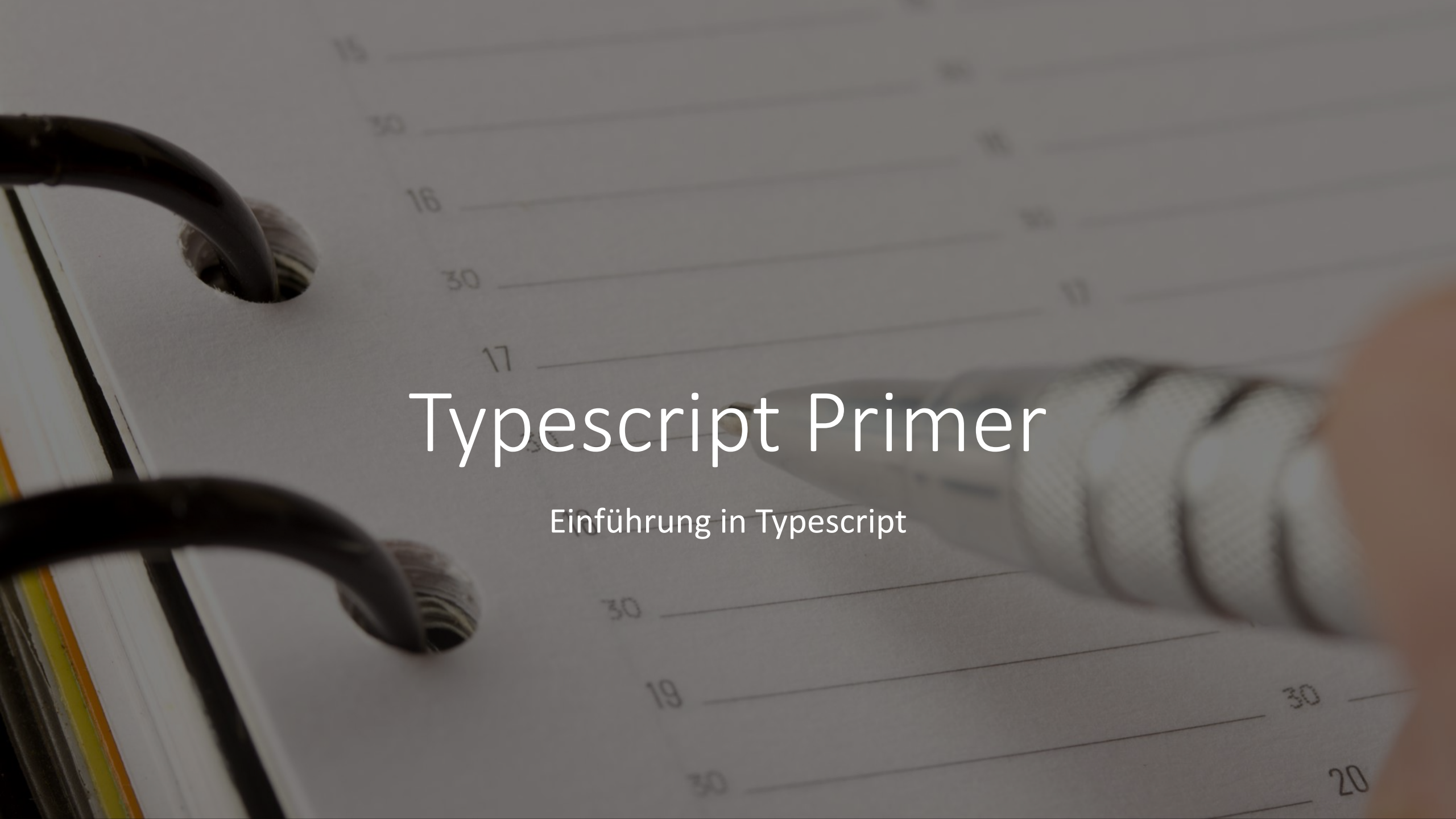




Typescript Primer

Einführung in Typescript

Einrichten von Typescript

Installation

Benötigte Komponenten

- TypeScript
 - `npm install typescript --save-dev`
- Oder global
 - `[sudo] npm install -g typescript`
- Lite-Server
 - `npm install lite-server --save-dev`
- Parallelisierung von npm-Kommandos
 - `npm install concurrently --save-dev`

Konfiguration des TypeScript-Compilers

- Initialisierung mit `tsc --init`
 - Erzeugt die Datei *tsconfig.json*
 - Darin werden alle möglichen Konfigurationen angelegt
 - wobei die allermeisten auskommentiert sind
- Für die folgenden Beispiele wird insbesondere der strict-Mode aktiviert

Scripts der package.json

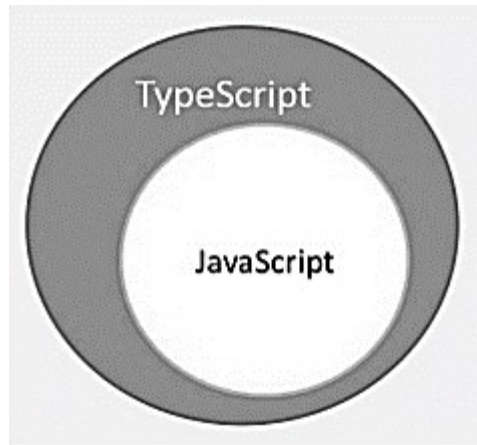
```
"scripts": {  
  "serve": "lite-server",  
  "compile": "tsc --outDir ./dist -p .",  
  "compile-watch": "tsc -w --outDir ./dist -p  
  .",  
  "test": "echo \"Error: no test specified\"  
&& exit 1",  
  "start": "concurrently \"npm run compile-  
watch\" \"npm run serve\""  
}
```

Benutzung

- Starten mit `npm start`
 - Startet den TypeScript-Transpiler
 - Startet den Lite Server mit Browser-Sync
 - Startet den Default-Browser und stellt die `index.html`-Seite dar
- Parallelisierung
 - Sämtliche Dateien mit der Endung `.ts` im Ordner `src` werden automatisch nach `dist` transpiliert
 - Änderungen der `.ts`-Dateien werden automatisch erkannt
 - Der Server aktualisiert den Browser mit den geänderten Informationen

TypeScript is just JavaScript

TypeScript is portable.



TypeScript supports other JS libraries

JavaScript is TypeScript.





OO Concepts



Supports Type Definitions



Strong static Type



Compilation

Type Hierarchy

any

build-in

user-defined

Spezielle Typen

- `null`
 - Eine Eigenschaft ist nicht gesetzt
- `undefined`
 - Eine Eigenschaft oder Funktion ist nicht vorhanden
 - Damit unterschiedlich zu `null`
- `any`
 - Eine untypisierte Variable, die jeden Wert zugewiesen bekommen kann
 - Damit ist bei Bedarf auch eine untypisierte Programmierung auch in TypeScript möglich
- `void`
 - Eine Funktion, die keinen expliziten `return`-Wert aufweist
- `never`
 - Der Rückgabotyp einer Funktion, die kein implizites oder explizites `return`-Statement aufweist
 - Endlose Ausführung oder garantiertes Werfen einer `Exception`

Variablen Deklaration

- `var [identifier] : [type-annotation] = value ;`
- `var [identifier] : [type-annotation] ;`
- `var [identifier] = value ;`
- `var [identifier] ;`

Typisierte Deklaration von Variablen

- Explizite Typisierung

```
let name : string
let state : boolean
```

- Type Inference

- Hier wird der Typ durch die Zuweisung eines Wertes definiert

```
let name = "Hello"
let state = true
```

- Contextual Type

- Auch bei Zuweisungen versucht TypeScript, untypisierte Deklarationen zu erkennen

```
window.onmousedown = function(mouseEvent) {
    console.log(mouseEvent.button); //<- Error,
};
```

Deklaration von Variablen mit const und let

- `let | const<name>`
 - `const name`
 - `let state`

Variable Samples

```
var message:string = "Hello Typscript";  
console.log(message);
```

```
function f() {  
    var message = "Hello, world!";  
    return message;  
}
```

Var-Scoping Macken

```
function weirdScoping(someFlag: boolean) {  
    if (someFlag) {  
        var x = 10;  
    }  
    return x;  
}  
  
weirdScoping(false);
```

Var-Scoping Macken 2

```
function sumOf(matrix: number[][]) {  
    var sum = 0;  
    for (var i = 0; i < matrix.length; i++) {  
        var row = matrix[i];  
        for (var i = 0; i < row.length; i++) {  
            sum += row[i];  
        }  
    }  
  
    return sum;  
}
```

Var-Scoping Macken 3

```
for (var i = 0; i < 10; i++) {  
    setTimeout(function() { console.log(i); }, 100 * i);  
}
```

Block Scoping

```
function someFunc(flag: boolean) {  
    let a = 0;  
  
    if (input) {  
        let b = a + 1;  
        return b;  
    }  
  
    // CompileError: 'b' doesn't exist  
    return b;  
}
```



Block Scoping 2

```
function foo() {  
  return a;  
}  
  
// undefined  
console.log(foo());  
  
let a;
```

Block Scoping 3

```
function sumOfWithLet(matrix: number[][])  
{  
    let sum = 0;  
    for (let i = 0; i < matrix.length; i++) {  
        var row = matrix[i];  
        for (let i = 0; i < row.length; i++) {  
            sum += row[i];  
        }  
    }  
  
    return sum;  
}
```

const

```
52  const someConst = 9;  
53  someConst = 10;
```

⊗ basic_syntax.ts 1 of 1 problem

Cannot assign to 'someConst' because it is a constant. ts(2588)



```
if (boolean_expression1) {  
  //some  
} else if (boolean_expression2) {  
  //someOther  
} else {  
  //elsedo  
}
```



```
var n:number = 1
while(n > 5) {
  //some
}

do {
  //some
}
while(n>5)
```

Number Datentyp

MAX_VALUE

MIN_VALUE

NaN

NEGATIVE_INFINITY

POSITIVE_INFINITY

Number Funktionen

- toFixed()

- `var numToFixed = new Number(123.456);`
- `console.log("numToFixed.toFixed() is "+numToFixed.toFixed());`
- `console.log("numToFixed.toFixed(2) is "+numToFixed.toFixed(2));`
- `console.log("numToFixed.toFixed(6) is "+numToFixed.toFixed(4));`

- toLocaleString()

- `var numTo = new Number(123.456);`
- `console.log( numTo.toLocaleString());`

String Funktionen

```
var str1 = new String( "The TS String" );  
var index = str1.indexOf( "String" );  
console.log("indexOf found String :" + index );  
  
console.log("(1,2): " + str1.substring(1,2));  
console.log("(0,8): " + str1.substring(0, 8));
```

Funktionen deklarieren

- function name (param1[:type], param2?[:type])

Varargs AKA Rest Parameters

```
function sumUp(...nums:number[]) {  
    var sum:number = 0;  
    for(var i = 0;i<nums.length;i++) {  
        sum = sum + nums[i];  
    }  
    console.log("sum of the numbers",sum)  
}  
  
sumUp(1,2,3)  
sumUp(1,1,1,1,1,1)
```

Default Parameter

- `function name(param1[:type],param2[:type] = default_value) { }`

Anonyme Funktionen

```
var res = function( [arguments] ) { ... }
```

```
( [param1, param2,...param n] )=>statement;
```

Funktionen überladen

1. SIGNATURES

```
function someOverload(n:number) : void;  
function someOverload(s:string) : void;
```

2. 1 BODY FOR ALL

```
function someOverload(x:any):void{  
  console.log(x);  
}
```

Container

- `array`
 - Eine Liste
- `tuple`
 - Eine feste Menge von anderen Basis-Typen
- `enum`
 - Eine feste Menge von Werten

```
var array_name[:data type] = [val1,val2...valn]
```

```
var nummern:number[];  
nummern = [1,3,5,7];
```

```
var secondEntry = nummern[1];
```

0

1

2

3

4

5

6





```
var nummern2:number[] = new Array(4)

for(var i = 0;i<nummern2.length;i++) {
    nummern2[i] = i * 10;
    console.log(nummern2[i])
}
```



Array Schnittstellen Auszug

- `concat(value1, value2, ..., valueN) : T[];`
- `indexOf(o) : number;`
- `lastIndexOf(o) : number;`

- `push(e1,e2,..en) : number;`
- `unshift(e1,e2,..en) : number;`
- `pop() : T;`
- `shift() : T;`

https://www.tutorialspoint.com/typescript/typescript_arrays.htm

concat()

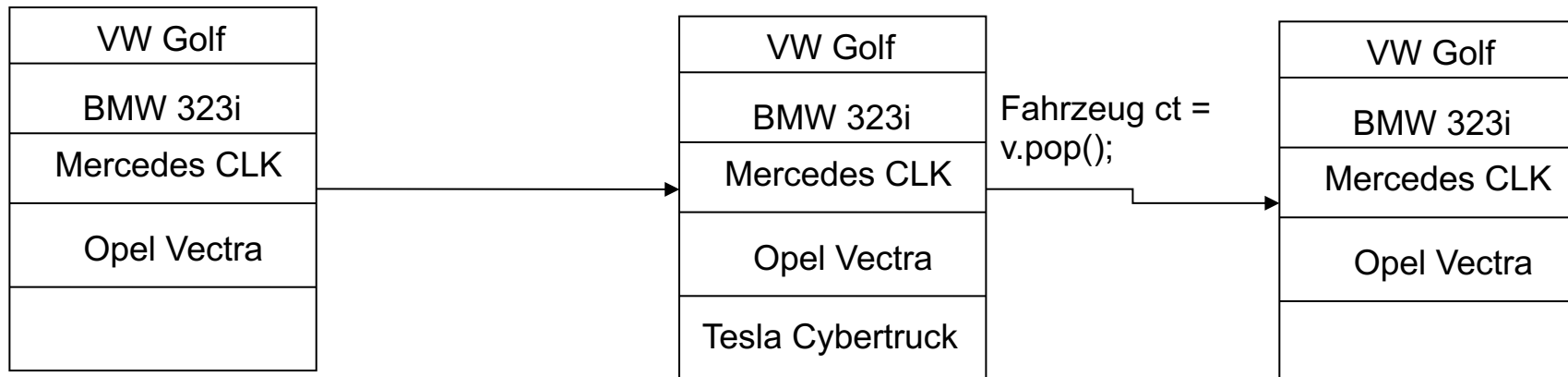
```
var chars:any[] = ["a", "b", "c"];
```

```
var nummern2 = [1, 2, 3];
```

```
var alphaNumeric = chars.concat(nummern2);
```

Pop und push

```
var lengthAfterPush = v.push(  
    new Fahrzeug("Cybertruck Dual Motor", "Tesla"));  
var fahrzeug:Fahrzeug = v.pop();
```



UNION TYPES

```
var someUnionValue:string|number  
someUnionValue = 42  
console.log("numeric value "+someUnionValue)  
someUnionValue = "42 as string"  
console.log("string value "+someUnionValue)
```



Schnittstellen - Interfaces

- Eine Schnittstelle beschreibt eine Menge von Signaturen und Attributen
- Externe *Sicht* auf eine Klasse
 - Verhalten der Klasse
 - Enthält in der Regel nicht alle Attribute/Methoden der Klasse
- Eine Klasse kann mehrere Interfaces *implementieren*
 - Muss alle in der Schnittstelle deklarierten Methoden implementieren
 - Schnittstellenmethoden sind de facto abstrakt

Enum

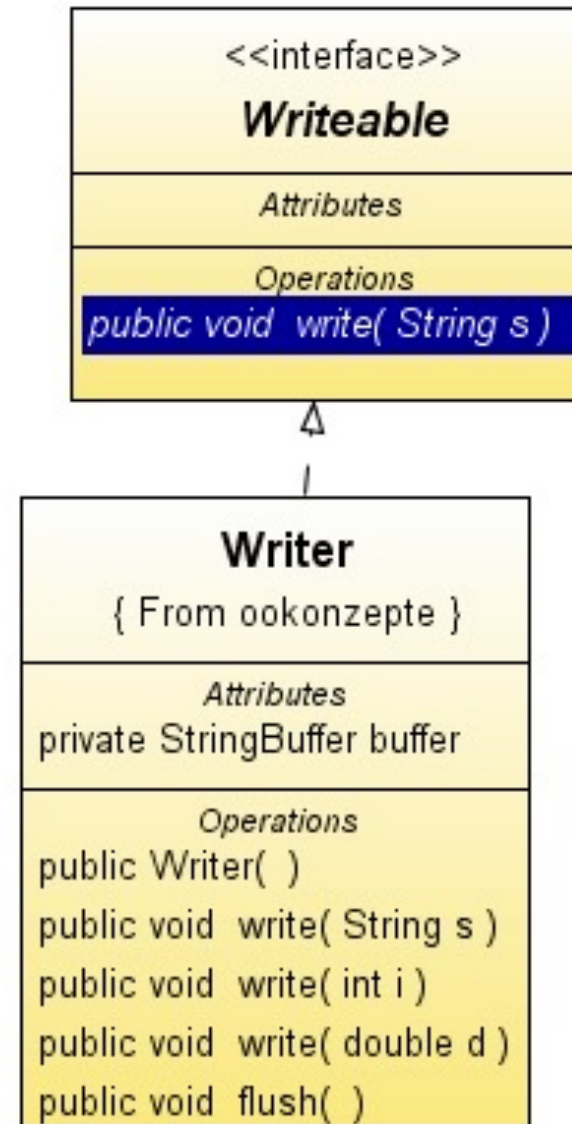
```
enum Direction {  
    Up = 1,  
    Down,  
    Left,  
    Right  
}
```

```
var dir: Direction = Direction.Up;
```

Nonsense Code

```
• function some(x: Direction) {  
• if (x != Direction.Up || x != Direction.Down)  
  {  
• // ~~~~~  
• // Error! This condition will always return  
  'true' since the types 'Direction.Up' and  
  'Direction.Down' have no overlap.  
• }  
• }
```

Beispiel Interface



Interface Beispiel

```
interface Schmutzfink{  
    name: string,  
    suhlen : () => string;  
}  
  
console.log(  
    {name:"Miss Piggy",  
      suhlen:():string => "Ich mag nicht schmutzig sein"}  
);
```

Interface Beispiel mit var

```
var schweinchenDick:Schmutzfink = {  
    name : "schweinchenDick",  
    suhlen: ():string => name+ " ist gerne im Dreck"  
}  
  
console.log(schweinchenDick.suhlen());
```

Inline Interface Def

```
function printName(named : {name:String}){  
    console.log(named.name);  
}  
printName(schweinchenDick);
```

Optional und Readonly

```
interface Animal{  
    readonly name: String,  
    weight?: number;  
}  
  
var missPiggy2 : Animal = { name: "Miss Piggy"};  
var missPiggy2_1 : Animal = { name: "Miss Piggy", weight: 100};
```

Klassen
deklarieren

```
class class_name {  
    //class scope  
}
```

Member von Klassen



Felder

Konstruktoren

Funktionen

Klassen Definition

```
class Schwein{  
    name : String;  
  
    constructor (name:String){  
        this.name = name;  
    }  
  
    gibLaut():String{  
        return "Ich bin " + this.name;  
    }  
}
```

Klassen: readonly

- `readonly`-Attribute sind möglich
- Verkürzter Konstruktor durch "Parameter properties"
 - `constructor(readonly attr:type)` deklariert und setzt ein Attribut

Sichtbarkeit

- Sichtbarkeiten von Exemplarvariablen und Methoden
 - public Zugriff von allen Klassen möglich (+)
 - protected Zugriff von Unterklassen (#)
 - private Zugriff nur innerhalb Klasse (-)



Sichtbarkeit Beispiel

```
class Schwein{  
private name : String;
```

Übung 1: Erstelle die Klasse Student, so dass sie wie hier arbeitet

```
let s: Student = new Student();
```

```
let label: string = s.getLabel(123456);
```

```
console.log(label); // "Student mit Matrikelnummer: 123456"
```

Übung 2: Erstelle einen Konstruktor,
so dass die Klasse wie hier arbeitet

```
let s: Student = new Student("Max", 123456);  
let label: string = s.getLabel();  
console.log(label); // "Student Max mit Matrikelnummer: 123456"
```

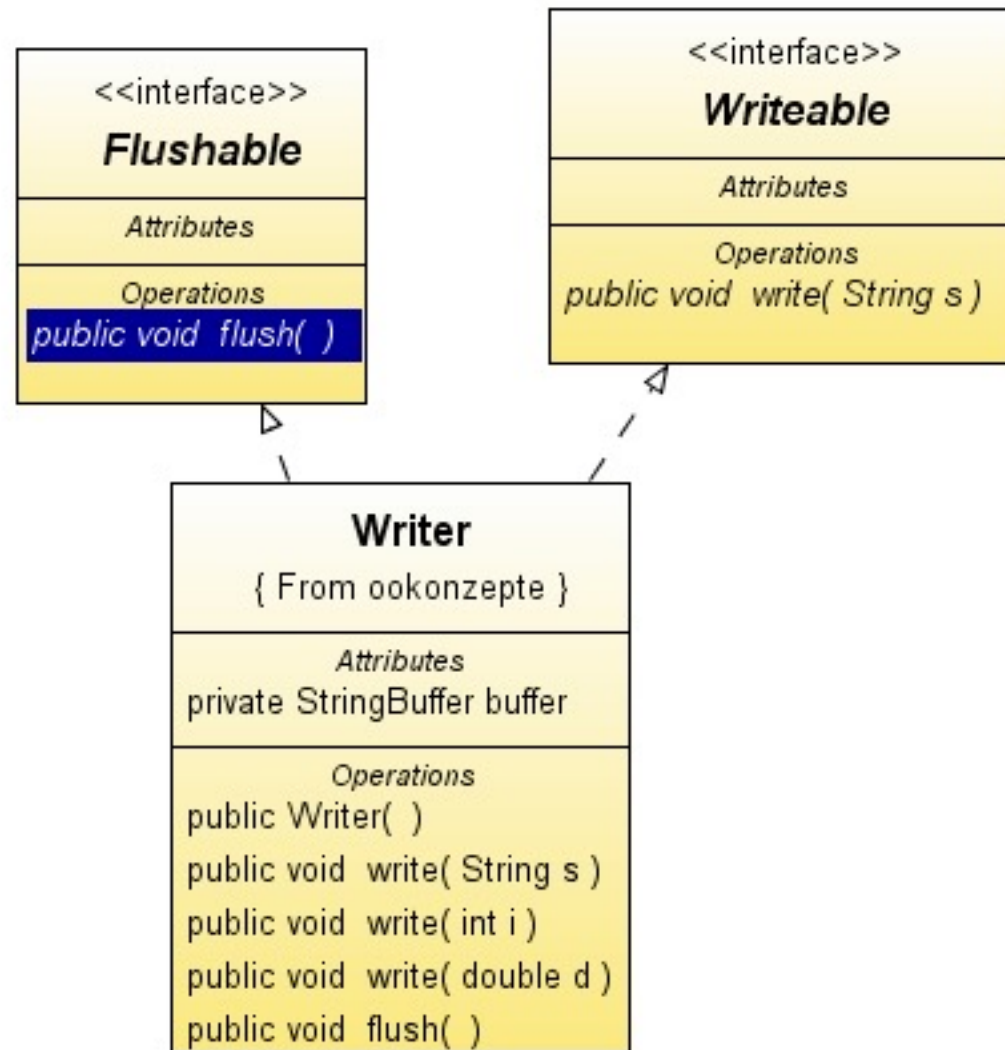
Getter Setter

```
private _name : string;

constructor (name:string){
    this._name = name;
}

get name():string{
    return this._name;
}

name(new_name:string){
    this._name = new_name;
}
```

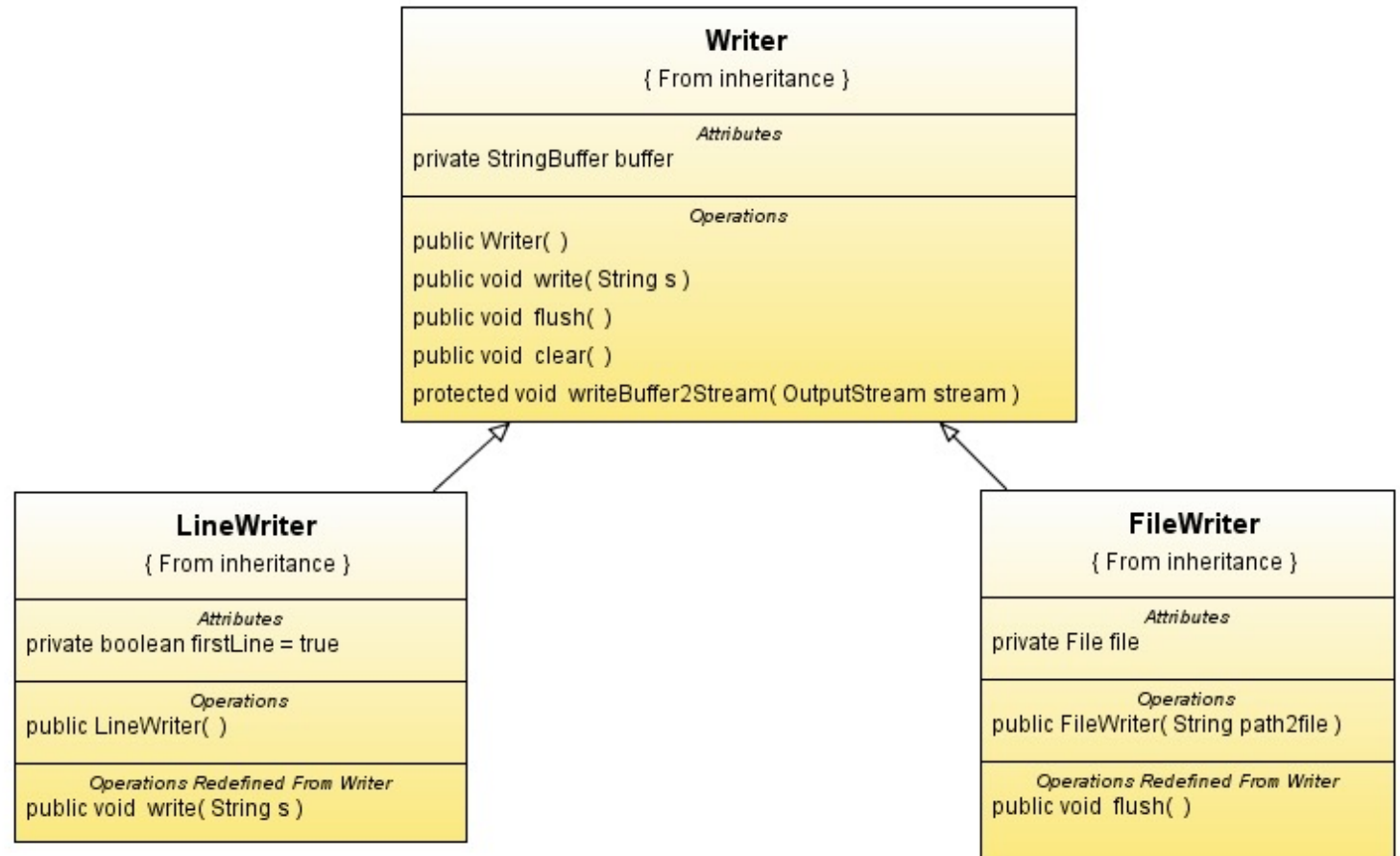


Beispiel

```
interface Schmutzfink{  
    suhlen : () => string;  
}
```

```
class Schwein implements Schmutzfink{  
    private name : string;  
    constructor (name:string){  
        this.name = name;  
    }  
  
    suhlen():string{  
        return "Ich bin " + this.name;  
    }  
}  
  
var miss_piggy : Schmutzfink = new Schwein("Miss Piggy");  
miss_piggy.suhlen();
```

Vererbung



Klassen: Vererbung

- Auch Klassen unterstützen das Konzept der Vererbung
- Methoden einer Klasse können auch abstrakt sein
 - Analog zu Definition einer Interface-Funktion
 - Eine Klasse, die mit `new` instanziiert werden soll darf keine abstrakten Methoden enthalten
- Ein Interface kann von einer Klasse erben
 - Allerdings darf die Klasse keine nicht-abstrakten Methoden enthalten
- Eine Klasse kann eine Schnittstelle implementieren
 - Schlüsselwort `implements`

```
class Schwein implements Schmutzfink{
    private name : string;
    constructor (name:string){
        this.name = name;
    }

    suhlen():string{
        return "Ich bin " + this.name;
    }
}

class Mastschwein extends Schwein{
    genmodifikation: number;

    constructor(name:string, genmodifikation:number){
        super(name);
        this.genmodifikation = genmodifikation;
    }
}
```

Klassen im Detail: Attribute und Methoden

- Methoden können überschrieben werden
 - Eine Subklasse implementiert die selbe Signatur einer Methode wie die Superklasse
 - Die Aufrufe von überschriebenen Methoden werden zur Laufzeit ausgewertet
 - Polymorphie
 - Der Zugriff auf eine Methode der Superklassen-Hierarchie ist mit der Referenz `super` möglich

Namespaces

Code Gruppen

Namespaces

- Namespaces gruppieren Deklarationen
 - Diese sind nur innerhalb des Namespaces direkt ansprechbar
 - Damit wird die Wahrscheinlichkeit von Namenskollisionen vermieden
- Deklarationen werden mit Hilfe des Schlüsselworts `export` anderen Namespaces zur Verfügung gestellt
- Aus einem anderen Namespace müssen die Variablen mit dem Namespace angesprochen werden
 - "Qualifizierte Namen"

Animal.ts

```
namespace Animals{
    export interface Animal{
        makeSomeNoise():void;
    }
}
```

Pig.ts

```
/// <reference path="Animal.ts" />
namespace Animals{
    export class Pig implements Animal{
        constructor(){
        }
        makeSomeNoise(): void {
            console.log("Oink");
        }
    }
}
```

Main.ts

```
/// <reference path="Animal.ts" />
/// <reference path="Pig.ts" />
```

```
let pig = new Animals.Pig();
console.log(pig.makeSomeNoise());
```

Weitere Exporte

- `let aPrivate = 2;`
-
- `export let publicA = 1;`
- `export let publicB = () => { };`
- `export function publicC() {`
- `// ...`
- `};`
-
- `export class PublicD {`
- `// ...`
- `}`

Named importieren

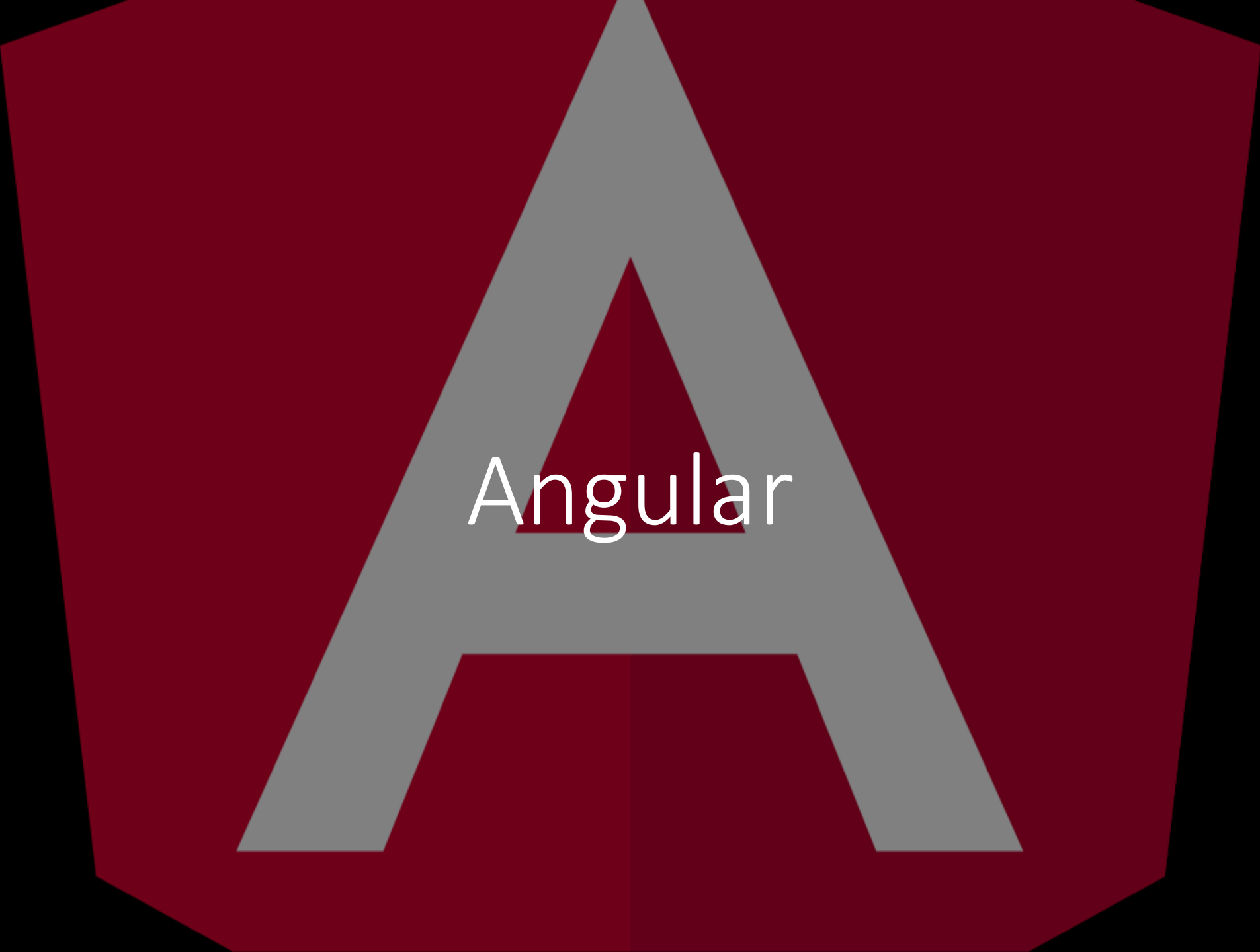
- **named exports** importieren
 - `import {publicA, publicB} from "./lib";`

Imports

- Alle import Statements sind *hoisted*
- import/export Statements dürfen nicht ...
 - variabel sein
 - conditional sein (if, case, etc.)
- Erlaubt statische Analyse
 - Bundling
 - Code Navigation in IDE

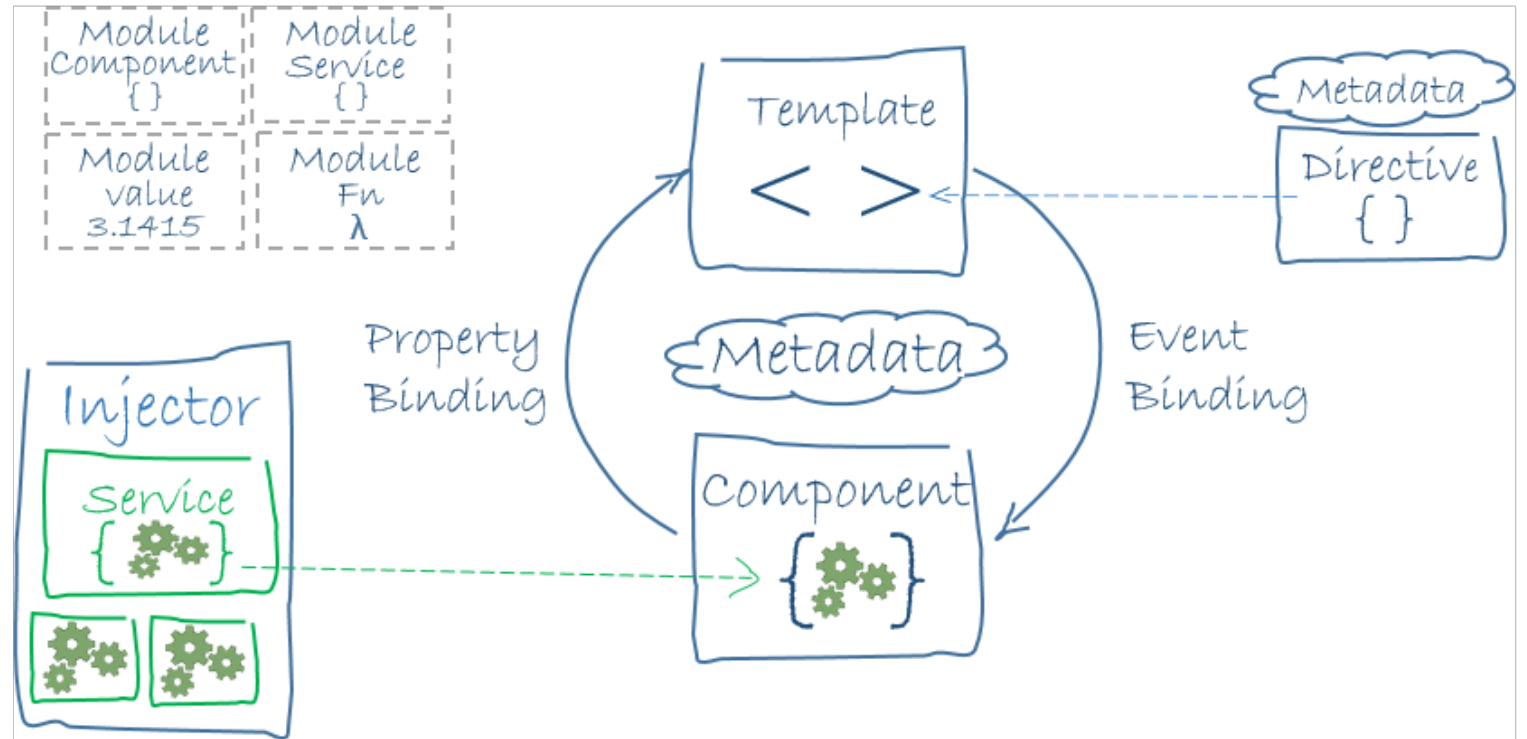
Übung 3: Module

- Lagere die Klasse Student in ein Module aus
 - Erstelle eine Datei student.ts
 - Verschiebe die Klasse dort hin und exportiere sie
- Importiere die Klasse in der ursprünglichen Datei

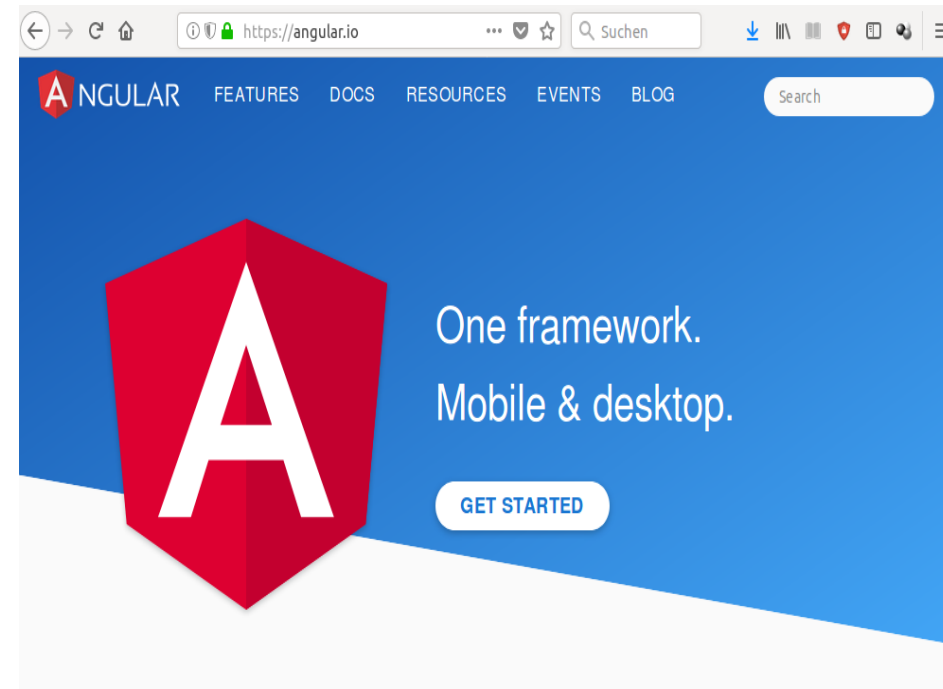
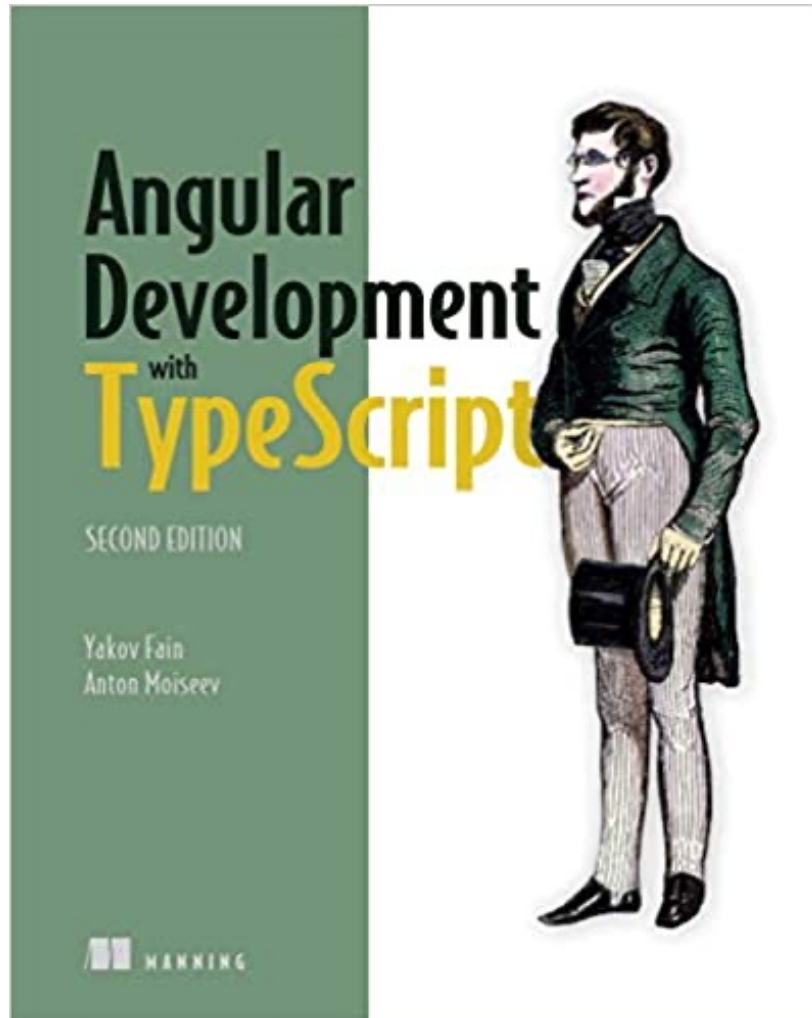
The Angular logo is centered on a black background. It features a large, stylized letter 'A' in a light gray color. This 'A' is set against a dark red, shield-like background that has a slightly irregular, hand-drawn appearance. The word 'Angular' is written in a white, sans-serif font, positioned horizontally across the middle of the gray 'A'.

Angular

Architektur Überblick



Literatur und Quellen





Installation und Setup

JavaScript- Grundinstallation

node und npm sind auf einem Entwicklerrechner zu installieren

- Näheres hierzu im Anhang

Damit steht ein ausgefeilter Buildprozess zur Verfügung

- Verzeichnisstruktur und Projekt-Organisation
- Automatische Transpilation
- Browser-Update bei Änderungen an den Quellen

Ein spezieller Editor ist nicht notwendig

- Empfohlen wird Atom oder ähnliches

Installation des Angular Command Line Interfaces

- `npm install -g @angular/cli`
- Damit ist der Angular-Projektassistent `ng` installiert
 - Anlegen eines neuen Projekts
 - `ng new <projektname>`
 - Erzeugen neuer Angular-Programmteile
 - Services
 - Direktiven
 - ...
- Freunde des Handmade Manifesto:
 - Das Anlegen eines Angular-Projektes kann natürlich auch ohne dieses Werkzeug erfolgen

Überblick über ng command line

- **Development server**
 - `ng serve` startet einen Dev-Server (Vite/esbuild) auf `http://localhost:4200/`
- **Code Scaffolding**
 - `ng generate component component-name` (standalone ist Default)
 - `directive|pipe|service|class|guard|interceptor|interface|enum|resolver`
- **Build**
 - `ng build` (Production ist Default, Application Builder mit esbuild)
- **Unit Tests**
 - `ng test` – neue Projekte ab v21 mit Vitest, Bestandsprojekte mit Karma/Jasmine
- **End-to-End Tests**
 - `ng e2e` – Runner per `ng add` hinzufügen, z. B. Cypress oder Playwright
- **Updates**
 - `ng update @angular/core @angular/cli` – inkl. automatischer Migrationen

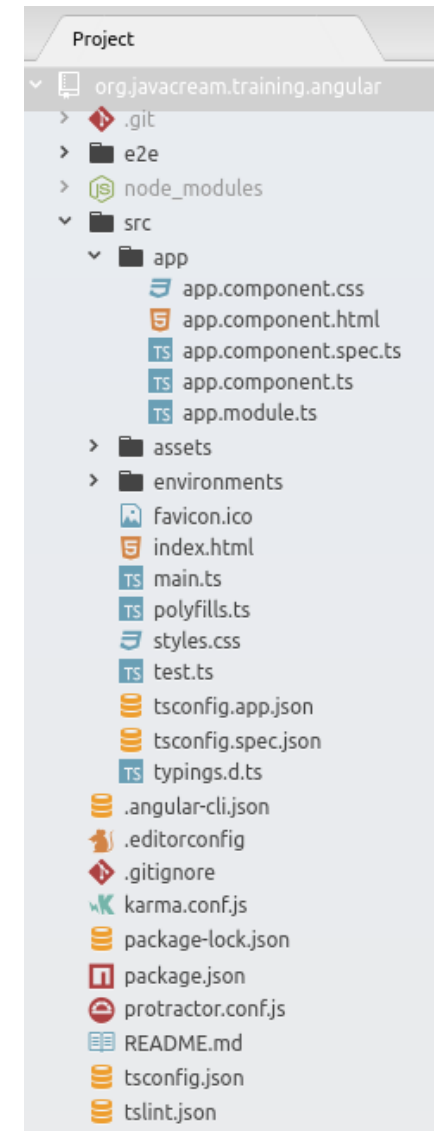


Ein erstes Projekt

Anlegen des Projekts

- `ng new org.somepackage.angular`
- Damit wird ein node-Projekt erzeugt
- Ebenso werden automatisch alle Abhängigkeiten installiert
 - Insbesondere TypeScript

Projektstruktur



Die
Anwendung
unter
localhost:4200



Welcome to app!



Here are some links to help you start:

- [Tour of Heroes](#)
- [CLI Documentation](#)
- [Angular blog](#)

Übung 4: Erstelle Initiales Projekt

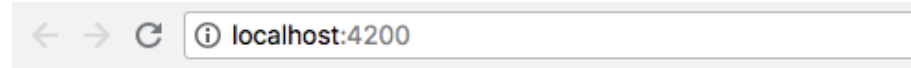
- Öffne ein Terminal
- Installiere, falls noch nicht geschehen, das Angular CLI global (siehe [Benötigte Software](#))
- Navigiere zu einem Ordner in dem das neues Projekt anzulegen werden soll
- Lege eine neues Projekt an: `ng new invoice-app`
- Wechsel in das erzeugte Verzeichnis `invoice-app`
- Führe `ng serve` aus
- Öffne Anwendung im Browser unter <http://localhost:4200>



Reload der App im Hintergrund bei Change

- `ng serve -o` (observe)
- Beispiel:
 - Ändern des Seitentitels sowie der Willkommens-Nachricht
 - `src/index.html`
 - `src/app/app.component.html`
 - `src/app/app.component.ts`

Die
aktualisierte
Seite



Welcome to app!



Here are some links to help you start:

- [Tour of Heroes](#)
- [CLI Documentation](#)
- [Angular blog](#)

Projektüberblick

Standalone Components

- Komponenten, Direktiven und Pipes sind standardmäßig standalone
 - standalone: true muss seit Angular 19 nicht mehr angegeben werden
- Kein NgModule zur Deklaration nötig
 - Jede Komponente deklariert ihre Abhängigkeiten selbst
- imports: [] direkt in der @Component-Annotation
 - Andere Komponenten, Direktiven, Pipes (z. B. RouterLink, FormsModule)
 - Für @if / @for ist kein CommonModule nötig
- Vorteile
 - Weniger Boilerplate, besseres Tree-Shaking, einfaches Lazy Loading
- NgModules: siehe Legacy-Kapitel am Ende (Übung 25)

Übersicht der Dateien eines Projekts

- `src/index.html`
 - Die Hauptseite
- `src/main.ts`
 - *Start der Anwendung per `bootstrapApplication` – ersetzt `app.module.ts`. Module siehe Projekt 25 (Legacy)*
- `src/app/app.component.ts`
 - Definition des dynamischen Anteils einer Komponente mit TypeScript
- `src/app/app.component.html`
 - Definition des HTML-Templates einer Komponente
- `src/app/app.component.css`
 - Stylesheet der Komponente
- `src/app/app.component.spec.ts`
 - Definition von Unit-Tests (Vitest bzw. Jasmine/Karma)

Referenz auf Bootstrap Component in Index-Seite

```
index.html x
1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <title>org.Javacream.Training.Angular</title>
6    <base href="/">
7
8    <meta name="viewport" content="width=device-width, initial-scale=1">
9    <link rel="icon" type="image/x-icon" href="favicon.ico">
10 </head>
11 <body>
12   <app-root></app-root>
13 </body>
14 </html>
15
```

Der
Einstiegspunkt
für Angular

Component-Annotation

- Eine Component ist eine TypeScript-Klasse mit der `@Component`-Annotation
 - Annotationen enthalten Metadaten, die der Klasse zugeordnet werden
 - Diese müssen von einem Framework interpretiert werden
- Annotationen sind damit so etwas wie statische Eigenschaften einer Klasse
 - Allerdings sind diese syntaktisch nicht Bestandteil der Klassendeklaration
 - Damit ergibt sich eine saubere Trennung
- Eine typische Component definiert
 - Den Selector als Bezug zum Template
 - Die Lokation des Templates
- Vollständige Liste unter
 - *<https://angular.io/api/core/Component>*

Aufgabe der Component

- Die Eigenschaften der Component-Klasse dienen als Schnittstelle zum HTML-Code
 - Werte werden über Interpolation oder Direktiven gebunden
 - Funktionen dienen als Event-Handler
- Bei Bedarf implementiert die Component-Klasse Lifecycle-Funktionen
 - `ngOnChanges`
 - `ngOnInit`
 - *<https://angular.io/guide/lifecycle-hooks>*

Beispiel: Component Klasse

```
import {ApplicationRef, Component, OnDestroy, OnInit} from "@angular/core";  
@Component({  
  selector: "app-root",  
  templateUrl: "./app.component.html",  
  styleUrls: ["./app.component.scss"],  
  animations: [fadeAnimation]  
})  
export class AppComponent implements OnInit, OnDestroy {  
  
  public title = "theFrontend";  
}
```

TypeScript-Import der
Angular-Core-Bibliothek

Referenz auf den
Einstiegspunkt

Referenz auf Template
und CSS

Definition einer
Variablen und
Wert-Zuweisung

Das HTML-Template einer Component

```
app.component.html x
1  <!--The content below is only a placeholder and can be replaced.-->
2  <div style="text-align:center">
3    <h1>
4      Welcome to the great {{ title }}!
5    </h1>
6    
9  </div>
10 <h2>Here are some links to help you start: </h2>
11 <ul>
12   <li>
13     <h2><a target="_blank" rel="noopener" href="https://angular.io/t
14   </li>
15   <li>
16     <h2><a target="_blank" rel="noopener" href="https://github.com/a
17   </li>
18   <li>
19     <h2><a target="_blank" rel="noopener" href="https://blog.angular
```

Referenzieren der
Component-Variable mit
Interpolation

Kompletter Ablauf

- Ablauf insgesamt
 - Aufruf bootstrapApplication mit Component (AppComponent)
 - Komponenten-Klasse hat Decorator
 - Komponenten-Decorator hat Feld selector
 - Selector wird in DOM aus index.html gesucht
 - Komponente wird instanziiert

Bootstrapping einer Standalone-App

```
// src/main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideZonelessChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';

bootstrapApplication(AppComponent, {
  providers: [
    provideZonelessChangeDetection(),
    provideRouter(routes),
    provideHttpClient()
  ]
}).catch(err => console.error(err));

// Kein AppModule mehr nötig – Legacy: siehe Übung 25
```

Einfache Erweiterungen

Mehrere Komponenten

- Auf einer Seite können auch mehrere Anwendungen dargestellt werden
- Diese sind komplett voneinander isoliert
 - Der Datenaustausch zwischen verschiedenen Komponenten wird später beschrieben

Export von Benutzerdefinierten Typen

- Eine Komponente kann auch eigene Datentypen definieren und Exportieren
 - Ein Export ist nicht notwendig, so lange der Typ nicht anderen Komponenten bekannt gemacht werden muss
- Ebenso können die Eigenschaften der Komponente komplexe Objekte sein
- Der Zugriff auf die Eigenschaften des komplexen Objekts erfolgen über die normale JavaScript-Syntax
 - Also über den `.`-Operator
 - Aber Vorsicht!
 - Die Deklarativen unterstützen nicht die komplette Syntax!
 - `if`
 - `for`
 - Deklarationen

Auflistungen

- Innerhalb der `{{ }}`-Interpolation wird das `for`-Konstrukt nicht unterstützt
- Allerdings gibt es eine spezielle `forEach`-Anweisung

```
<ul>
  @for(person of people; track person.id){
    <li >
      <span>{{person.firstname}}</span> {{person.lastname}}
    </li>
  }
</ul>
```

UI-Programmierung

Data Binding

- Template nicht statisch sondern dynamisch
- Komponenten-Klasse stellt Daten bereit
 - Felder / Instanz Variablen
- Template zeigt Daten an
 - Interpolation mit {{ expression }}
 - Gibt Ergebnis der Expression als String aus
 - Zugriff nur auf Instanz-Variablen der eigenen Klasse
 - Template-Variablen

Die ngModel-Direktive

- Bisher wurden die Werte der Komponente mit der `{{ }}`-Interpolation dargestellt
- Dieses Binding ist dynamisch (!)
 - Änderungen der Werte werden in der Oberfläche dargestellt
- Eigenschaften der Komponente können auch bidirektional gebunden werden
- Dazu dient die Direktive `ngModel`
 - Notwendig hierzu ist der Import von `FormsModule` aus `@angular/forms` im `imports`-Array der Komponente

Data Binding: Beispiel

```
import { Track } from "../tracks/track.model";

@Component({
  selector: "music-app",
  template: `
    <h1>Music App</h1>

    Music App has {{tracks.length}} tracks to play `
})

export class MusicAppComponent {
  public tracks: Track[];

  constructor() {
    this.tracks = [
      new Track("Sound of Silence"), new Track("I Wish"),
      new Track("Over My Shoulder"), new Track("Take Five")
    ];
  }
}
```

Übung 5: Databinding

- Erstelle ein Datenmodell
 - Lege eine neue Datei `src/app/customer/customer.model.ts` an.
 - Erstelle und exportiere darin ein Klasse Customer mit `id`, `firstname`, `lastname`, `street`, `zip`, `city`.
 - Vergebe für jeden erzeugten Customer eine eindeutige ID.
- Erweitere AppComponent
 - Ersetze im Template den dynamischen Titel gegen statisches HTML (`<h1>Invoice Manager App</h1>`).
 - Importiere in der AppComponent Klasse die Klasse Customer.
 - Stelle für das Template eine Liste mit Customer bereit (Feld + Initialisierung im Konstruktor).
 - Zeige im Template die Länge der Liste an.

Direktiven

- Allgemeines Konzept
 - Komponenten
 - Strukturelle Direktiven
 - Attribut-Direktiven
- *Direktiven* meint meist strukturelle und Attribut-Direktiven
 - Wie Komponenten Teil des UI
 - Unterschied zu Komponenten: kein eigenes Template

Direktiven

- Direktiven sind Erweiterungen von HTML, die vom Angular Framework gesucht und interpretiert werden
 - Während des Bootstrap-Vorgangs wird die Startseite der Anwendung nach Direktiven analysiert
 - Erweiterung durch
 - HTML-Elemente
 - Attribute
- Was genau beim Auftreten einer Direktive passiert hängt von der Direktiven-Implementierung ab
 - `ngModel`
 - Data Binding
 - `@for`
 - Iteration

Beispiel

```
@for (track of tracks; track track.id) {  
  <li>{{track.name}}</li>  
} @empty {  
  <p>Keine Tracks gefunden.</p>  
}
```

Direktiven des Angular-Frameworks

- Angular unterscheidet die folgenden Direktiven
 - Components sind Direktiven!
 - sie werden an ein HTML-Element gebunden
 - Structural Directives
 - Modifizieren das DOM der Anwendung
 - `for` oder `if` sind Beispiele hierfür
 - Attribute Directives
 - Ändern die Darstellung oder das Verhalten eines HTML-Elements
 - `ngStyle` als Beispiel
- Vollständige Liste unter
<https://angular.io/api?status=stable&type=directive>

Built-in Control Flow: @if, @for, @switch

```
@if (customers.length > 0) {  
  <p>{{ customers.length }} Kunden</p>  
} @else {  
  <p>Keine Kunden</p>  
}  
  
@for (customer of customers; track customer.id) {  
  <li>{{ customer.firstname }}</li>  
} @empty {  
  <li>Keine Kunden gefunden.</li>  
}  
  
@switch (customer.status) {  
  @case ('active') { <span>aktiv</span> }  
  @default { <span>inaktiv</span> }  
}
```

// Kein Import von CommonModule nötig, bessere Typprüfung und Performance

Noch ein paar Beispiele

```
@if (track.playable) {  
  <button>play</button>  
} @else {  
  <span>nicht verfügbar</span>  
}
```

Früher (Legacy, CommonModule nötig)

```
<div *ngIf="track.playable">...</div>
```

Attribut Direktiven

- Keine Veränderung der DOM Struktur

```
<form [formGroup]="form">  
    <input type="email" formControlName="mail"/>  
</form>
```



Übung 6

- Erweitere das Template von AppComponent
 - Zeige eine Liste aller Customer an



Child Components

Divide and conquer



Goals

- Listen Darstellung in eigene Komponente auslagern
 - Komplexere Darstellung ermöglichen
 - Wachsendes UI aufteilen
 - Wiederverwendbarkeit erreichen

Verschachtelung

- Gesamtes UI ist aus Baum von Komponenten aufgebaut
 - "Anwendung" ist Top-Level-Komponente
 - Komponenten können geschachtelt werden
- Vater- und Kind-Komponente sind isoliert
 - Kein Zugriff auf Daten des anderen
 - Daten müssen an Kind übergeben werden

Komponente als Direktive einbinden

- Komponenten sind auch Direktiven (eine Art)
- Direktiven (alle Arten) müssen bekannt gemacht werden
 - Per imports: [] in der @Component-Annotation
- Nur bekannt gemacht Direktiven werden ausgeführt

Mittels ng-generate

- Angular CLI legt neue Komponenten an
 - Klasse + alle benötigte Annotation
 - Html-Template
 - Leeres Styling
 - Test
- ng generate component *packageName/component-name*

Übung 7: Komponenten schachteln

- Erstelle eine Komponente CustomerListComponent
 - Generieren per ng generate component customer/customer-list
- Binde die neue Komponente ein
 - Importiere CustomerListComponent im imports-Array von AppComponent
 - Verwende CustomerListComponent im Template von AppComponent

Input einer Component

- Components können über das Template verschachtelt werden
 - Innerhalb des Templates wird ein Root-Element einer weiteren Angular-Component definiert
- Der Kind-Controller kann Input-Parameter definieren
 - Dazu wird eine Eigenschaft mit der `input`-Signal versehen
- Diese werden im Template gesetzt

Beispiel: Input einer Child-Component

```
<ul>
  @for(person of people; track person.id) {
    <li><people-detail [person]="person"></people-detail></li>
  }
</ul>
```

```
@Component({
  selector: 'people-detail',
  imports: [],
  templateUrl: './people-detail.html',
  styleUrls: ['./people-detail.css'],
})
export class PeopleDetail {
  person = input.required<Person>();
}
```

```
<h3>{{person().firstname}} {{person().lastname}}</h3>
```

AppComponent:
Neues Root-Element für
die Kind-Component

PeopleDetail
Component: Angabe des
Root-Elements

Deklaration einer Input-
Eigenschaft

Inputs

- Daten können an Komponente übergeben werden
 - *Property Binding* über [propertyName]="expression"
 - [propertyName] Property in der "inneren" Komponente *release-card*
 - expression Property in der "äußeren / aufrufenden" Komponente
- Data-Binding in Angular immer unidirektional
 - Objekt veränderbar
 - Referenz bleibt außen immer gleich

Übung 8: Inputs

- Erweitere die Komponente CustomerListComponent
 - Nimm als Input ein Array von Customer Objekten entgegen
 - Verschiebe die bisherige Darstellung aus AppComponent in das Template von CustomerListComponent (Text und Liste, nicht die h1 Überschrift)
- Passe die Verwendung von CustomerListComponent in AppComponent an
 - Lösche den entsprechenden Teil des Templates
 - Übergebe app-customer-list die Daten per Property-Binding

Events

Output aus Components

Event Binding

- Data-Binding unidirektional
 - Daten von außen nach innen
 - Wie kommen Informationen von innen nach außen?
- Komponenten können Events nach außen geben
 - *Event Binding* mit (eventName)="statement"
 - Zuweisung oder Funktionsaufruf
 - Zugriff auf das Event immer über \$event möglich

Event Handler

- Im Gegensatz zum Standard-Eventmodell im Browser können beliebige Funktionen mit Parametern als Handler aufgerufen werden
- Die Event-behandelnde Funktion ist Bestandteil der `Component`

Event Handler: Beispiel

```
export class UiAppComponent {  
  //...  
  selectionList: string[] = ["Item1", "Item2", "Item3"];  
  setItem(item: string) {  
    this.data.boundData = item;  
  }  
}  
  
<ul>  
  @for (item of selectionList; track item) {  
    <li (click)="setItem(item)">{{item}}</li>  
  }  
</ul>
```

Übung 9: Events

- Reagiere in CustomerListComponent auf Klicks auf die Listen-Elemente
 - Rufe eine Methode (listItemClicked) in der Component-Klasse auf
 - Übergebe das Dom-Event (\$event vom Typ MouseEvent) und das Training des geklickten Listen-Eintrags
 - Logge den Namen des Customer und das Event auf die Konsole
z.B. mit `console.log(event, customer.firstname)`

Output -- Events aus Komponenten

- Komponenten können Events nach außen werfen
- Eigenschaft wird mit der Funktion `output<T>()` angelegt
 - Typ-Parameter gibt Typ des Events an
 - Rückgabetyp ist `OutputEmitterRef<T>`
- Events werden über `emit(wert)` gefeuert
- Legacy: `@Output()` mit `new EventEmitter<T>()`

Output Beispiel

```
import { Component, input, output } from "@angular/core";

@Component({
  selector: "release-card", template: `<h3>{{release().name}}</h3>`
})
export class ReleaseCardComponent {
  release = input.required<Release>();

  // we use domain object as event, real world: ReleaseLikedEvent
  releaseLiked = output<Release>();

  likeRelease() {
    this.releaseLiked.emit(this.release());
  }
}
```

Nutzende Komponente

```
<div class="releases">  
  @for (release of artist.releases; track release.id) {  
    <release-card  
      [release]="release"  
      (releaseLiked)="onReleaseLiked($event, release)">  
    </release-card>  
  }  
</div>
```

Methode im aufrufendem Controller
NICHT in der Release-Card Komponente

Lokaler State mit Signals

```
import { Component, computed, signal } from '@angular/core';
export class AppComponent {
  selectedCustomer = signal<Customer | undefined>(undefined);
  hasSelection = computed(() => this.selectedCustomer() !== undefined);

  newCustomerSelected(customer: Customer) {
    this.selectedCustomer.set(customer); // oder .update(old => ...)
  }
}

<!-- Template: Signals werden wie Funktionen gelesen -->
<customer-details [customer]="selectedCustomer()" />
// Signals + zoneless: Angular weiß genau, was sich geändert hat
```

Übung 10: Output

- Erweitere CustomerListComponent
 - Erstelle in CustomerListComponent ein Output `customerSelected = output<Customer>()`
 - Emitte in `CustomerListComponent#listItemClicked` `customerSelected` ein Event
 - Erstelle in AppComponent ein Event-Binding für `customerSelected` an CustomerListComponent
 - Speichere in AppComponent den zuletzt selektierten Customer in einem `signal()`
- Erstelle eine Komponente CustomerDetailsComponent
 - Generieren per `ng generate component customer/customer-details`
 - Erstelle einen Signal-Input `customer = input<Customer>()` und zeige im Template dessen Properties an
 - Importiere CustomerDetailsComponent im imports-Array von AppComponent
- Verbinde die beiden Komponenten in AppComponent
 - Binde CustomerDetailsComponent in AppComponent ein (parallel zu CustomerListComponent)
 - Reiche das selektierte Customer an CustomerDetailsComponent weiter (`[[customer]="selectedCustomer()"`)
 - Zeige in AppComponent die Detail-Ansicht nur an, wenn ein Customer selektiert wurde (NPE vermeiden)

AppComponent
selectedCustomer =
signal<Customer | undefined>()

[customer2Show]="selectedCustomer()"

(customerSelected)="newCustomerSelected(\$event)"

CustomerListComp

DetailComponent
customer2Show =
input<Customer>()

{{customer2Show()?.name}}



Expression Language

Was wir fast alles schon kennen

Interpolation	{{ expression }}
Property Bindings	[propertyName]="expression"
Event-Bindings	(eventName)="statement"
Two-Way-Data-Binding	[(ngModel)]="assignable expression"
In Control Flow	@if(expression)

Expression

Template Operatoren

- Pipe Operator |
- Elvis Operator als Safe Navigation ?.

Evaluation Kontext

- Globaler Namensraum nicht zugreifbar
- this --> Instanzvariablen der @Component Klasse
- Eigene Template Variablen

Was geht nicht

- new Operator
- Bit-Operatoren | und &
- Inkrement bzw. Dekrement

Best Practice

Halte Expressions **einfach und lesbar**

Komplexe Logik → in den Component (am besten als **computed Signal**)

Safe Navigator

- `<p>{{bean?.name}}</p>`

Falls bean null oder undefined dann keine Exception sondern Leerstring

- Auch mehrfach
 - `<p>{{bean?.complexProperty?.name}}</p>`



Performance Impact

Schlecht, wird bei jeder Change Detection gecalled

```
@if (isActive(user)) { ... }
```

Besser

```
// Im Component
```

```
activeUser = computed(() =>  
this.isActive(this.currentUser()));
```

```
// Im Template
```

```
@if (activeUser()) { ... }
```

One Way Property Bindings bei Images

```
<img [src]="release.cover">
```

```
<button [disabled]="!track.playable"></button>
```

Event Binding

- One-Way nach oben im Komponenten-Baum
- `$event` als Handle auf das Event-Objekt
- Ziele
 - Standard-DOM-Events
 - `<input type="text" (keyup)="logInput($event.target.value)">`
 - Custom-Events an Komponenten
 - `<release (releaseLiked)="highlight($event)"></release>`

Style Bindings

- Class-Bindings
 - `<div class="selected">`
 - `<div [class.selected]="isSelected(track)"></div>`
- Style-Bindings
 - `<img [style.visibility]="release.cover ? 'visible' : 'hidden'">`
- Alternativ: `[class]="{ selected: isSelected(track) }"` bzw. `[style]-` Bindings (`ngClass/ngStyle` nicht mehr nötig)

Data Binding in Formularen

- Ziel:
 - Formularfelder an Properties binden
- Manuelles Synchronisieren würde bedeuten
 - `<input [value]="bean.name">`
 - `(input)="bean.name=$event.target.value">`
- Zwei-Wege-Data-Binding mit ngModel
 - `<input [(ngModel)]="bean.name">`
- *Merke: Banana in a Box [()]*

Deferrable Views: @defer

```
@defer (on idle) {  
  <customer-details [customer]="selectedCustomer()" />  
} @placeholder {  
  <div>Lade Kundendetails...</div>  
} @loading (minimum 500ms) {  
  <p>Lädt...</p>  
} @error {  
  <p>Fehler beim Laden</p>  
}
```

// Trigger: on idle | on viewport | on interaction | on hover | on timer(2s) | when bedingung

// Die Komponente landet in einem eigenen Bundle und wird erst bei Bedarf geladen

Übung 11: Bindings auf Images und mehr

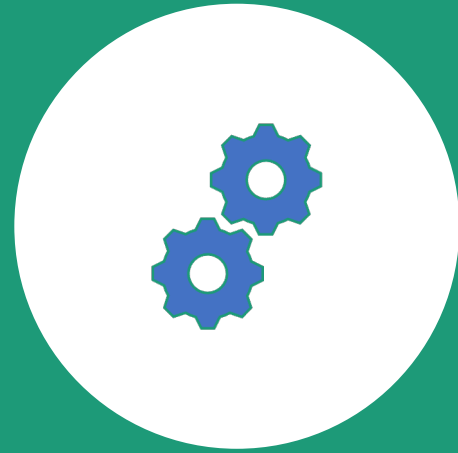
- Erweitere das Datenmodell
 - Füge der Klasse Customer eine optionale Eigenschaft imageUrl: string hinzu
 - Nehme imageUrl im Konstruktor als Parameter entgegen und setze das Feld
- Passe AppComponent an
 - Kopiere aus der Musterlösung (11_image) die Bilder aus src/assets/images/customer
 - Setze beim Erzeugen der Testdaten bei manchen Customer ein Bild.
Pfad z.B. assets/images/customer/carl.svg
- Erweitere die Darstellung in CustomerDetailsComponents
 - Zeige das Bild mit einer festen Größe an, falls vorhanden
 - Blende das Bild aus, falls kein Bild vorhanden
 - Optional: Lade CustomerDetailsComponent in AppComponent per @defer (on idle) mit @placeholder

Übung 12: Class Binding

- Erweitere das Datenmodell
 - Lege in Customer ein Feld discontinued: boolean an, das per Konstruktor gesetzt wird
 - Erzeuge in AppComponent mindestens ein Customer, an den nichts mehr geliefert wird
 - Setze für diesen Customer discontinued auf true
- Passe die Darstellung in CustomerListComponent an
 - Setze an nicht länger gewünschte Customer eine Klasse
 - Stelle nicht länger gewünschte Customer anders dar (über Klasse an Element und CSS in customer-list.component.css, z.B. ausgegraut oder durchgestrichen)
- Hebe in der Liste das selektierte Customer hervor
 - Speichere in CustomerListComponent den zuletzt selektierten Customer in einem signal()
 - Prüfe am ob der aktuelle Customer das selektierte ist und setze entsprechend eine Klasse

Übung 13: Formulare

- Biete in der Detail-Ansicht die Möglichkeit an, den Namen eines Customers zu verändern
 - Importiere FormsModule aus @angular/forms im imports-Array von CustomerDetailsComponent
 - Erstelle ein input mit ngModel



Services

Eigenschaften eines Services



Services sind TypeScript-Klassen



Diese werden ausschließlich
vom Angular-Framework
instanziert

Diese Aufgabe übernimmt
der Angular-Context



Benötigt eine Component den
Zugriff auf einen Service, so
wird dieser automatisch
gesetzt

Dieses Verfahren heißt
Dependency Injection



Insgesamt stellt Angular mit dem Service-
Mechanismus ein Dependency-Injection-Framework
zur Verfügung



Services? Was

Sehr allgemeines Konzept

- Irgendwelche Funktionalität kapseln
- Nicht UI spezifisch
- Implementierungsdetails verstecken

Einsatzmöglichkeiten

- Höhere Abstraktion schaffen (REST)
- Integration anderer Bibliotheken (WebSocket)
- Datenhaltung, Datenzugriff, Caching, Logik, ...

Können Abhängigkeit zu anderen Services haben

Aufgaben eines Services

- Standard-Services des Angular-Frameworks stellen nützliche Routinen zur Verfügung
 - Beispielsweise wird die Client-Server-Kommunikation über den `HttpClient`-Service realisiert
- Eigene Services werden in der Anwendung häufig benutzt, um in einer sauberen Architektur ein Datenmodell zu definieren
 - Dieses wird dann von allen Components konsistent benutzt
 - Services ermöglichen damit eine Inter-Component-Kommunikation, die unabhängig von Input-Parametern sind
 - Wesentlich leichter wartbar

Ein simpler Service

- Die Erzeugung eines Services kann das Angular-CLI übernehmen
 - `ng generate service <ServiceName>`
- Ein Service selbst ist eine `@Injectable`-Klasse
 - `@Injectable({ providedIn: 'root' })` – anwendungsweiter Singleton, tree-shakeable
- Nutzung in jeder anderen Angular-Klasse per `inject()` (oder Konstruktor)
 - Components
 - Andere Services, Guards, Interceptors
- Alternativen zur Registrierung
 - providers in `bootstrapApplication` (`main.ts`)
 - providers der `@Component`-Annotation
 - <https://angular.dev/guide/di>

Methode in Services

```
import {Artist} from "../artist.model";
```

```
export class ArtistService {  
  getAllArtists(): Artist[] {  
    return [];  
  }  
}
```

API Stabilität

- Service versteckt Details über Datenhaltung vor Komponenten
- API stabil, Datenquelle austauschbar
- Best Practice: Mock-Daten parallel zu Service in eigene Datei ablegen

```
import {Artist} from './artist.model';  
import {ARTISTS} from './artist.mock';
```

```
export class ArtistService {  
  getAllArtists(): Artist[] {  
    return ARTISTS;  
  }  
}
```

Objektnetze in OO- Systemen

Aufbau der Netze komplex.

Testbarkeit der Komponenten wird eingeschränkt.

Änderbarkeit und Wartbarkeit nur bedingt gegeben.

Favor Delegation over inheritance.

Verschiedene Möglichkeiten Netze aufzubauen.

Assignment by Third Party

```
public static void main(String[] args)
{
    MyService service = new MyService();
    service.setDAO(new MyDAO());
}
```

Creating itself - DIRECT

```
public MyService()  
{  
    this.myDao = new MyDao();  
}
```

Factory- Pattern

```
public MyService()  
{  
    this.myDao = Factory.getMyDao();  
}
```

Dependency Injection

- Objekte werden zur Laufzeit „injiziert“.
- Objekte werden passiv, statt sie zum Beispiel mit einer Factory aktiv zu erzeugen.
- Abhängigkeiten sind explizit an der Schnittstelle durch entsprechende set-Methoden zu erkennen.
- Es können beliebige andere Objekte zugewiesen werden(z.B. Mocks / an Bedürfnisse angepasste Objekte).
- Sehr flexibel wenn Objekte nur noch von Interfaces abhängen.
- Externe Konfiguration der Beziehungen zwischen Objekten.
- Objekte bekommen ihre abhängigen Objekte statt sie selbst zu suchen.

Beispiel: Service

```
1  import { Injectable } from '@angular/core';
2
3  @Injectable()
4  export class PeopleModelService {
5
6      constructor() { }
7      people: Person[] = [new Person("Gärtner", "Hans")]
8
9  }

```



```
providers: [
    PeopleModelService
],

```

```
export class ServiceAppComponent {
    constructor(readonly peopleModel: PeopleModelService ){

    }
    people:Person[] =  this.peopleModel.people
}

```

@Injectable macht diese Klasse dem Angular-Context bekannt

Im Modul oder der @Component-Annotation

Dependency Injection in einer Component

DI Konfig

- Woher weiß Angular was zu injizieren ist?
- Muss konfiguriert werden: Providers
 - Injector -> Provider -> Instance
- Konfiguration direkt am Service

```
@Injectable({ providedIn: 'root' })
```

```
export class TrackService { }
```

- oder global in main.ts

```
bootstrapApplication(AppComponent, { providers: [ TrackService ] });
```

- Wie sieht ein Provider aus?

Wie geht DI in Angular

- Direkte Abhängigkeit
 - einfach nur die Klasse angeben: [ConsoleLogger]
 - Kurz für [{ provide: ConsoleLogger, useClass: ConsoleLogger}]
- Allgemein: Provider-Objekt
 - provide: Unter welchem Namen steht es zur Verfügung -> Klasse oder InjectionToken
 - Ein Rezept, wie es erzeugt werden soll -> useClass, useValue, useFactory oder useExisting

```
bootstrapApplication(AppComponent, {  
  providers: [  
    { provide: ConsoleLogger, useClass: ConsoleLogger }, // same as shortcut  
    { provide: Logger, useExisting: ConsoleLogger } // different token, same instance  
  ]  
});
```

Moderne Dependency Injection mit inject()

```
import { Component, inject } from '@angular/core';

@Component({ selector: 'app-root', templateUrl: './app.component.html' })
export class AppComponent {
  private readonly customerService = inject(CustomerService);
  customers = this.customerService.getAll();
}

// Früher: constructor(private customerService: CustomerService) {}

// inject() funktioniert auch in Funktionen (Injection Context):
export const authGuard: CanActivateFn = () => inject(AuthService).isLoggedIn();

// Vorteile: weniger Boilerplate, einfache Vererbung, funktionale Guards & Interceptors
```

Kontextbasierte Injektion

- Injektoren bilden einen Baum
 - Jede Komponente ein Child-Injector vom Injector der Vater-Komponente
 - Instanzen sind Singletons in einem Injektor
- Provider für Child-Injektoren können an Komponenten definiert werden
 - Child-Injector kann Provider des Vaters überschreiben / ergänzen
 - providers Property in @Component Decorator
- Einsatzmöglichkeiten
 - Auflösen von Namenskonflikten
 - Unterschiedliche Konfiguration (z.B. Log-Level)

Übung 14

- Erstelle einen Service CustomerService
 - Generieren per ng generate service customer/customer
- Erstelle Mock-Daten
 - Erstelle eine Datei src/app/customer/customer.mock.ts
 - Exportiere darin ein Array von Customer (kopieren aus AppComponent)
- Verwende im Service die Mock-Daten
 - Importiere die Mock-Daten aus customer.mock
 - Erstelle in CustomerService eine Methode getAll(): Customer[]
 - Gebe darin die Mock-Daten zurück
- Verwende den Service noch nicht!

Übung 15

- Lasse CustomerService in AppComponent injizieren
 - Stelle den CustomerService über `@Injectable({ providedIn: 'root' })` zur Verfügung (Default von ng generate)
 - Injiziere CustomerService in AppComponent per `inject(CustomerService)`
- Verwende CustomerService in AppComponent um die Daten zu beziehen
 - Lösche den Code zum Erzeugen der Test-Daten in AppComponent
 - Rufe `CustomerService#getAll` auf um Daten zu bekommen

Komponenten Lebenszyklus

- Komponenten haben einen Lebenszyklus
 - **Aufruf des Konstruktors**
 - **ngOnChanges**
 - **ngOnInit**
 - ngDoCheck
 - ngAfterContentInit
 - ngAfterContentChecked
 - ngAfterViewInit
 - ngAfterViewChecked
 - [ngDoCheck und ggf. ngOnChanges]*
 - **ngOnDestroy**

Was soll das?

- Bestimmte Informationen stehen erst später zur Verfügung
 - ID des Trainings beim Routing
 - Kind-Komponenten
- Instanziierung und Initialisierung trennen
 - Gut für Testbarkeit
- Achtung: keine Lifecycle-Hooks für Services

Best Practices: OnInit

```
import { Component, OnInit, inject } from "@angular/core";
@Component({
  selector: "music-app", template: `<h1>Music App</h1>`,
  providers: [ ArtistService ]
})
export class MusicAppComponent implements OnInit {
  private readonly service = inject(ArtistService);
  public artists: Artist[] = [];
  ngOnInit() {
    this.artists = this.service.getAll();
  }
}
```

Übung 16: OnInit

Initialisiere die Daten in AppComponent im ngOnInit Hook statt im Konstruktor

Implementiere das Interface OnInit aus @angular/core

Verschiebe das Laden der Daten aus dem Konstruktor in die Hook-Methode

Pipes

Daten für UI aufbereiten

Allgemein

- Daten-Transformation für das UI
 - Kleine, wiederverwendbare Operationen für das UI
 - An Funktionale Programmierung angelehnt
 - Daten und Konfiguration per Parameter
 - Pipe verändert ursprünglichen Wert nicht
- Pipes werden an Expression mit | angehängt
 - `{{ property | json }}`
- Pipes können per : konfiguriert werden
 - `{{ release.releaseDate | date:'dd.MM.yyyy' }}`
- Pipes können verkettet werden
 - `{{ release.releaseDate | date | uppercase }}`

Was sind Pipes?

- Pipes formatieren das Ergebnis beispielsweise einer Interpolation
 - `{{dateOfBirth | date}}`
 - Hier wird das Geburtsdatum als Datum formatiert dargestellt
 - Pipes sowie der Operator `|` sind aus der Linux-Welt übernommen
- Vordefinierte Pipes umfassen
 - `date[:format[:timezone[:locale]]]`
 - `currency[:currencyCode[:display[:digitInfo[:locale]]]]`
 - `lowercase`
 - Vollständige Liste und Dokumentation unter [*https://angular.io/api?status=stable&type=pipe*](https://angular.io/api?status=stable&type=pipe)

Decimal Pipe

- Formatierung von Zahlenwert
- Parameter: Anzahl Nachkommastellen
`{{ track.size | decimal:2 }}`

Date Pipe

- Für Datum-String, Date-Object, Timestamp-Number
- Nimmt als Parameter Shortcut (z.B. short) oder Format-String entgegen
- Kurzschreibweise: Zusammenspiel mit locale des Browsers
 - `{{ release.releaseDate | date:'short' }}`
 - `{{ release.releaseDate | date:'dd.MM.yyyy' }}`

Übung 17: Pipe

- Erweitere das Datenmodell
 - Lege in Customer ein Feld / Konstruktor-Parameter lastOrderDate vom Typ Date an
- Passe die Mock-Daten an
 - Setze für jeden Customer ein unterschiedliches Datum (new Date(year, month - 1, day))
 - Für mind. 1 Customer weniger als 1 Woche zurück
 - Setze für nicht länger gewünschte Customer natürlich ein Datum in der weiteren Vergangenheit
- Erweitere das Template von CustomerDetailsComponent
 - Gib das Datum lastOrderDate formatiert aus
 - Probiere verschiedene Formattierungen aus (Pattern, Shortcuts)

Pipes sind Klassen

- Pipes sind Klassen
 - Implementieren Interface PipeTransform aus @angular/core
 - Werden mit @Pipe Decorator aus @angular/core markiert

```
import {Pipe, PipeTransform} from "@angular/core";
@Pipe({ name: "hasLikes" })
export class HasLikesPipe implements PipeTransform {
  transform(withLikes: {likes :number}): boolean {
    return withLikes.likes > 0;
  }
}
```

Verwendung von Pipes in Komponenten

```
import { DatePipe } from '@angular/common';
import { HasLikesPipe } from './has-likes.pipe';
@Component({
  selector: 'release-card',
  imports: [ HasLikesPipe, DatePipe ],
  template: `@if (release() | hasLikes) { <span></span> }`
})
export class ReleaseCardComponent { ... }
// Pipes sind standalone – Import direkt in der verwendenden Komponente
// Legacy: declarations/exports im NgModule (siehe Anhang)
```

Übung 18: Eigene Pipe

- Schreibe eine eigene Pipe `currentActiveCustomers`
 - Generieren per `ng generate pipe customer/current-active-customers-pipe`
 - Implementiere `PipeTransform` mit der Methode `transform(date: Date, daysInPast = 7): boolean`
 - Gib `true` zurück, falls der Customer innerhalb der letzten `x` Tage bestellt hat (Parameter `soon`)
`const timeDiff = Date.now() - date.getTime();`
 - `const maxTimeDiffInMS = daysInPast * 1000 * 60 * 60 * 24;`
 - `const isActive = timeDiff < maxTimeDiffInMS;`
- Wende die `currentActiveCustomers` Pipe auf das `lastOrderDate` Datum in `CustomerDetailsComponent` an (Pipe im `imports`-Array importieren)
 - Verpacke die Ausgabe des `lastOrderDate` Datums in einen `div`-Container
 - Setze an diesem `div` eine Klasse, wenn der Customer aktiv ist (`[[class.active]]= "..."`)
 - Übergebe z.B. 5 (Tage) als Parameter `daysInPast` an `currentActiveCustomers`
 - Stelle über CSS mithilfe der gesetzten Klasse `Customer`, die `active` sind, anders dar (z.B. rote Schrift)

nextRun:Date

Training.ts

nextRun

training.js

„id“:4,
"nextRun":“2
023-12-23“

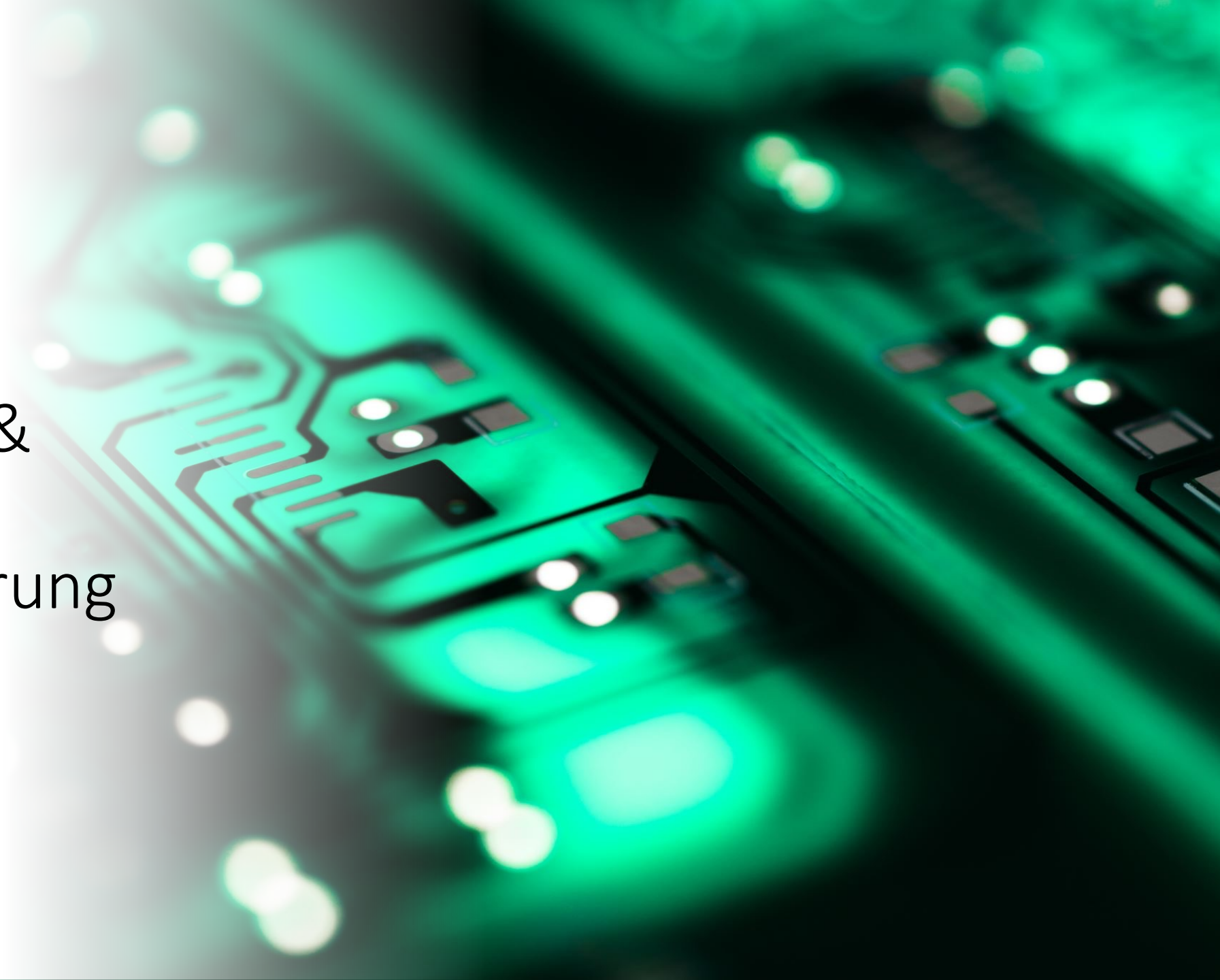
training.json

nextRun:
“2023-12-23“

Transform(value: Date)

takePlacesoonPipe.ts

Asynchrone & Reaktive Programmierung



Welche Eigenschaft hat die Variable X

```
var x = 1;  
console.log(x);
```

```
var x = [1, 2, 3];  
x.forEach(alert);
```

Und hier?

```
asyncFunction(function callback(x) {  
  console.log(x);  
});
```

Thread

t2

calculationLongRunning

t1

console.log(x);

Zeit

Threading im Browser

- Javascript per Definition Single-Threaded
- Event-Loop Thread mit Queue, die vom Interpreter abgearbeitet wird
- Beispiel: Ajax Aufruf
 - JavaScript startet HTTP Request
 - Interpreter wartet nicht bis Server geantwortet hat, UI würde einfrieren
 - Interpreter arbeitet andere Sachen ab (Event-Loop)
 - Nach Response, wird weiterer Code in Event-Loop-Queue eingestellt
 - Code läuft erst wenn in Event-Loop an der Reihe

Callback API

- Asynchroner Funktion wird Callback als zusätzlicher Parameter mitgegeben
- Callback wird aufgerufen sobald Ergebnis bereit steht
- Probleme
 - Keine standardisierte API (Wie werden Fehler behandelt?)
 - Mehrmalige oder spätere Nutzung kompliziert ("Callback-Hell")
 - Abhängigkeit von mehreren asynchronen Ergebnissen sehr schwierig zu handhaben

Lösung sind Promises

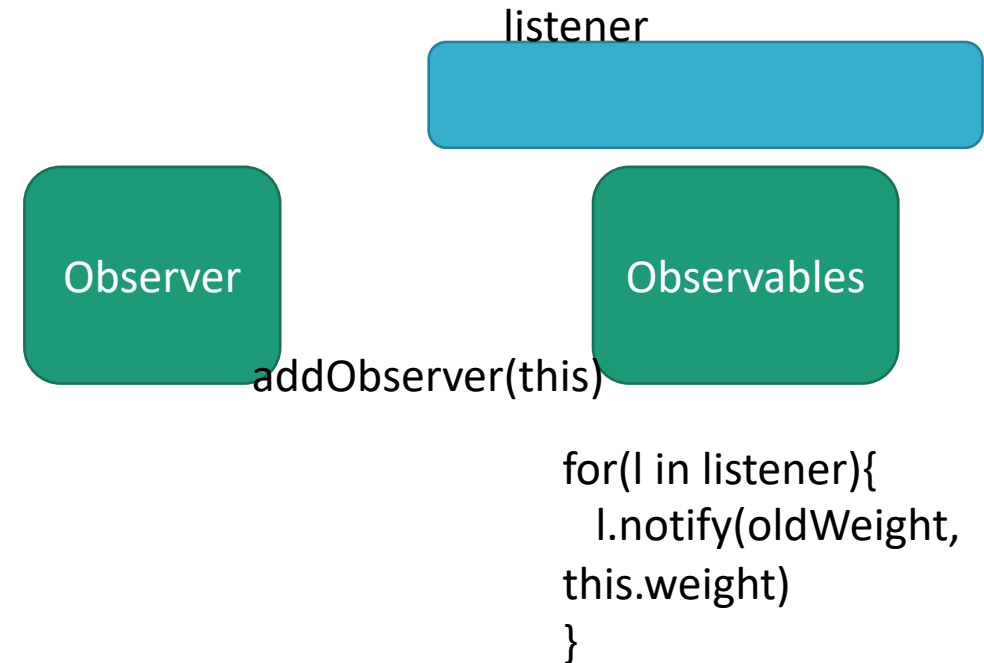
- Stellvertreter für Ergebnis, das zunächst unbekannt ist (z.B. http-Aufruf)
- Asynchronen Funktionen geben Promise zurück, kein Callback als Parameter
 - Können erfüllt oder abgelehnt werden
 - Bieten API für success- und error-Funktionen
- Promise kann genau ein Mal aufgelöst werden
 - → *Nicht geeignet für mehrere asynchrone Werte*

Promise Beispiel

```
// call to async function returns a promise
const promise = doSomethingAsynchronous();
promise.then(
  (result) => { return filter(result) }, // success handler
  (error) => { } // error handler (optional)
).then((result) =>);
```

Observable als Promise++

- Observables sind Streams von Events und Daten
 - Aber auch Fehlern
- --a---b-c---d---X|
 - -- Zeitline
 - | Endsignal
 - a,b .. Ereignisse
- Schönere und interaktive Visualisierung
- <https://rxmarbles.com>



Methoden für Zwischenverarbeitung

- Häufig genutzte Methoden:
 - map für Transformation
 - debounce für Verzögerung
 - filter zum Aussortieren
- Jede Transformation gibt wieder Observable zurück

--a---b-c---d---|

map to alphabet index

--1---2-3---4---|

filter > 2

-----3---4---|

Observables sind lazy

- Observables brauchen Abnehmer
 - `forEach(dataHandler)`: Promise
 - `subscribe(dataHandler, errorHandler?)`: Subscription
 - Handler: Funktionen mit Seiteneffekten
- Intermediate Transformationen werden nur ausgeführt wenn Abnehmer da ist

--a---b-c---d---|

map to alphabet index

--1---2-3---4---|

filter > 2

-----3---4---|

subscribe console.log

Intermediate vs Terminal

intermediate



- ☐ filter
- ☐ map
- ☐ sorted
- ☐ distinct
- ☐ limit
- ☐ peek

terminal



- ☐ count
- ☐ foreach
- ☐ sum
- ☐ min/max
- ☐ reduce
- ☐ findFirst

Unsubscribe Nicht vergessen

- subscribe() gibt Subscription zurück
- Subscription hat unsubscribe() Methode
- Wenn completed, werden alle intern alle Handler gelöscht

Nutze ngOnDestroy für unsubscribe

Modern: takeUntilDestroyed() aus @angular/core/rxjs-interop oder toSignal() – unsubscribe automatisch

Automatisches Unsubscribe

```
{{ aPromise | async }}
```

```
{{ anObservable | async }}
```

```
<trainingList [trainings]="releaseObservable$ | async"></trainingList>
```

Training[]

Observable<Training[]>

Alternative ohne async-Pipe: toSignal() (siehe RxJS Interop)

Reactive Extensions – RxJS

- Entwickelt von MS für verschiedene Sprachen
- Implementierung von Observables
 - Wird in Angular verwendet
 - <http://reactivex.io/>

RxJS Interop: toSignal()

```
import { toSignal } from '@angular/core/rxjs-interop';

export class AppComponent {
  private readonly customerService = inject(CustomerService);
  // Observable -> Signal, Subscription wird automatisch beendet
  customers = toSignal(this.customerService.getAll(), { initialValue: [] as Customer[] });
}

<!-- Template: keine async-Pipe nötig -->
<customer-list [customers]="customers()" />
// Gegenrichtung: toObservable(signal)
// RxJS für asynchrone Streams (HTTP, Events) – Signals für den Template-State
```

Übung 19: Async CustomerService

- Verpasse CustomerService eine Observable basierte API
 - Lege im Service ein ReplaySubject als Feld an
 - Rufe im Kontruktor am Subject next mit den Mock-Daten auf
 - Lasse getAll das Observable<Customer[]> des Subject zurückgeben
- Passe alle Aufrufe an (toSignal(), alternativ subscribe oder async-Pipe)
 - Eigentlich nur:
 - AppComponent: `customerMockList = toSignal(customerService.getAll(), { initialValue: [] })`
 - app.component.html: `<customer-list [customers]="customerMockList()">`

Routing und Navigation

Arbeitsweise

- Routen definieren Pfade der Anwendung
- Jede Route verbindet einen Pfad mit einer Component
 - Auch Redirects können definiert werden
 - Die Pfade unterstützen Platzhalter
- Die Routen werden in `app.routes.ts` definiert und per `provideRouter(routes)` in `main.ts` für die gesamte Applikation bereitgestellt
 - dazu werden Klassen importiert
 - `import { Routes, provideRouter } from '@angular/router'`
- Routen-Pfade müssen nicht disjunkt sein
 - Der erste Treffer eines Pfads wird benutzt
 - Damit sind in der Routen-Definition speziellere Pfade vor allgemeinen zu platzieren

Features von Routen

- Deep Linking über URL
- Browser Historie (Vor / Zurück)
- Links über symbolische Namen, nicht über hart codierte URLs
- Pfad- oder Hash-Navigation
- Hierarchischer Aufbau, nicht zwingend absolute Routen

Routendefinition

- Route
 - Relativer path (ohne führenden Slash)
 - component für die Anzeige

```
import { Routes } from '@angular/router';
```

```
export const routes: Routes = [  
  { path: 'release', component: AllReleasesComponent },  
  { path: 'artist', component: AllArtistsComponent }  
];
```

Ablauf

- URL des Browsers wird überprüft / auf Änderungen überwacht
- Passende Route wird gesucht
- Komponente wird angezeigt



Verlinken von Routen

- Dazu gibt es die `routerLink`-Direktive
- Die anzuzeigende Seite wird in einem `router-outlet`-Element angezeigt

```
@Component({  
  selector: "music-app",  
  template: `  
    <h1>Music App</h1>  
    <router-outlet></router-outlet>  
  `,  
})  
export class MusicAppComponent {}
```

Default / Fallback Route

- Keine gültige Route gefunden?
 - Einstieg über URL ohne Route
 - ungültige Route
- Fallback konfigurierbar
 - path mit **
 - redirectTo: auf anderen Pfad
 - Achtung: Reihenfolge der Routen spielt eine Rolle,
 - **Fallback als Letzte**

Routenkonfiguration

- `provideRouter(routes, ...features)` erzeugt die Provider für den Router-Service
- Aufruf einmalig in `bootstrapApplication (main.ts)`
- Features als optionale Funktionen
 - `withComponentInputBinding()` – Routen-Parameter als Inputs
 - `withHashLocation()`, `withDebugTracing()`, `withPreloading(...)`
- Lazy Loading per `loadComponent / loadChildren`
- Legacy: `RouterModule.forRoot / forChild` im `AppModule` (siehe Anhang)

Beispiel: Routen-Definition

```
// app.routes.ts
export const routes: Routes = [
  { path: 'path1', component: Component1 },
  { path: 'lazy', loadComponent: () => import('./lazy.component').then(m =>
m.LazyComponent) },
  { path: '', redirectTo: '/index', pathMatch: 'full' },
  { path: '**', component: PageNotFoundComponent }
];

// main.ts
bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes, withDebugTracing()) // <-- debugging purposes only
  ]
});
```

Location-Strategy

- LocationStrategy festlegen
 - PathLocationStrategy (default)
 - Bessere SEO Indizierung
 - HashLocationStrategy: default AngularJS 1
 - Simpler
- Verwendung von PathLocationStrategy
 - `<base href="/">` als erstes Element im head der index.html
- `import { provideRouter, withHashLocation } from '@angular/router';`
- `import { routes } from './app/app.routes';`
- `bootstrapApplication(AppComponent, {`
- `providers: [ provideRouter(routes, withHashLocation()) ]`
- `});`

Übung 20: Routen für Listenansicht

- Bereite die Anwendung auf Routing vor
 - Erstelle eine Datei `src/app/app.routes.ts`
 - Definiere und exportiere eine Konstante des Typs `Routes` (`export const routes: Routes = []`)
 - Registriere die Routen in `main.ts` per `provideRouter(routes)`
 - Importiere `RouterOutlet` im `imports`-Array von `AppComponent`
- Erstelle eine Route für die Listen-Ansicht
 - Füge in `app.routes.ts` eine Routen-Konfiguration für `path: "customer"` und `component: CustomerListComponent` hinzu
 - Erstelle auch eine `Redirect-Route` für `path: "**"` mit `redirect` auf `customer`
 - Lösche `CustomerListComponent` und `CustomerDetailsComponent` aus dem `Template` von `AppComponent`
 - Füge in das `Template` von `AppComponent` ein `router-outlet` ein
- Passe `CustomerListComponent` an die Veränderungen an
 - Daten können nicht mehr per `Input` übergeben werden, da keine direkte Verwendung
 - Lade in `CustomerListComponent` die Daten vom `Service` (`inject()` + `toSignal()`)
 - Kein `ngOnDestroy` nötig – `toSignal()` beendet die `Subscription` automatisch
 - Lösche das Laden der `customers` aus `AppComponent`

Pfadparameter für dynamische Information

Problem:

- Routen Konfiguration zur Entwicklungszeit
- Somit: Keine statische Route pro Datensatz möglich

Lösung:

- Dynamische Routen mit Parametern
- Routen Konfiguration kann in Pfadvariablen enthalten

Rest-Parameter vs
URL-Parameter

- Parameter in Route für Pflichtparameter
- URL-Parameter für optionale Parameter

Beispiel Pfadparameter

```
{  
  path: 'artist/:id',  
  component: ArtistDetailsComponent  
}
```

Details zu den Routen

- Modern: Routen-Parameter, Query-Parameter und Route-Data direkt als Input der Komponente
 - `provideRouter(routes, withComponentInputBinding())`
 - `id = input<string>()` in der Ziel-Komponente
- Alternativ: `ActivatedRoute` per `inject()` – Zugriff auf die Routen-Definition, die zum Aufruf geführt hat
 - `url, paramMap, queryParams, data, ...`
- Router-Events werden an interessierte Listener delegiert
 - `NavigationStart, RoutesRecognized, NavigationEnd, ...`
- Details unter <https://angular.dev/guide/routing>

Beispiel: Component Input Binding

```
// main.ts
```

```
provideRouter(routes, withComponentInputBinding())
```

```
// app.routes.ts
```

```
{ path: 'artist/:id', component: ArtistDetailsComponent }
```

```
// artist-details.component.ts
```

```
export class ArtistDetailsComponent {
```

```
  private readonly service = inject(ArtistService);
```

```
  id = input<string>();      // wird aus :id befüllt
```

```
  artist = computed(() => ...); // reagiert auf Änderungen von id()
```

```
}
```

Weiterverarbeitung von Routenparametern

```
export class ArtistDetailsComponent {  
  private readonly service = inject(ArtistService);  
  id = input.required<string>();  
  artist = toSignal(  
    toObservable(this.id).pipe(  
      map(id => parseInt(id)),  
      switchMap(id => this.service.getById(id))  
    )  
  );  
}  
  
// Legacy: this.route.params.pipe(map(...), switchMap(...)).subscribe(...)
```

Navigation im html

1.<a [routerLink]="['/artist']">Artists

2.<a [routerLink]="['/artist', artist.id]">Some Artist

3.<a [routerLink]="[artist.id]">Artist with ID

- Absolute Links mit führendem Slash (1,2)
- Zur aktuellen Route relative Links ohne führenden Slash (3)

Navigation im Code – Component Klasse

```
private readonly router = inject(Router);  
save() {  
    // Navigate back to list  
    this.router.navigate(['artist']);  
}
```

Übung 21: Routing mit Parametern

- Erstelle eine Route für die Detail Ansicht
 - Konfiguriere eine Route für `/customer/:id` die auf die `CustomerDetailComponent` verweist
 - Erweitere `CustomerService` um eine Methode `getById : Observable<Customer>`
 - Lese in `CustomerDetailsComponent` den Routen-Parameter per Signal-Input `id = input<string>()` aus (`withComponentInputBinding()` in `main.ts`)
 - Hole das Customer für die Anzeige per ID vom Service
- Verlinke die Listen-Ansicht mit der Detail-Ansicht
 - Importiere `RouterLink` im `imports-Array` von `CustomerListComponent`
 - Füge den Listen-Elementen einen `routerLink` hinzu
 - Navigiere im `routerLink` zur Detail-Ansicht für das Element
- Verlinke die Detail-Ansicht mit der Listen-Ansicht
 - Füge im Template von `CustomerDetailsComponent` einen Link zurück zur Liste ein
 - Verwende einen Button mit einem Click-Handler
 - Navigiere im Handler per `router.navigate (Router per inject())`



Formulare und Eingabefelder

Bearbeiten von Daten

Rückblick auf Standard html

- Formular (form) einfacher Aufbau
 - Mit Feldern (input, select, ..)
 - FieldSet für visuelle Gruppierung
- Validierung mittels HTML 5 (teilweise)
- Posten der Daten mittels Pagereload oder AJAX

Formulare in Angular

Arten

- Template-Driven
- Model-Driven

Interne Forms API

- Binding (bzw. ChangeDetection)
- Validierung
- Aggregation

Formulare, Gruppen, Felder

- Ein Formular = Gruppe + Submit-Aktion
- Eine Gruppe enthält Liste von Feldern
 - Zustand abhängig von enthaltenen Feldern
 - z.B: Gruppe invalide, falls min. 1 Feld invalide
- Feld ist ein Wert + weitere Zustände
 - dirty, touched
 - valid

Formulare Bauen mit ReactiveFormsModule

- Model-Klassen um Formular zu beschreiben
 - FormGroup und FormControl
 - Gruppen bilden Baum mit Feldern als Blätter
 - Formular selbst ist eine FormGroup
 - Seit Angular 14 streng typisiert: `FormGroup<{ name: FormControl<string> }>`

HowTo

```
import { FormGroup, FormControl, ReactiveFormsModule } from "@angular/forms";
```

```
@Component({ imports: [ReactiveFormsModule], ... })
```

```
export class ReleaseFormComponent implements OnInit {
```

```
  release = input.required<Release>();
```

```
  form = new FormGroup({
```

```
    name: new FormControl("", { nonNullable: true }),
```

```
    date: new FormControl<string | null>(null)
```

```
  });
```

```
  ngOnInit() {
```

```
    this.form.patchValue({ name: this.release().name });
```

```
  }
```

```
}
```

```
<form [formGroup]="form" (ngSubmit)="onSubmit()">
```

```
  <label>Name:</label>
```

```
  <input type="text" [formControl]="form.controls.name">
```

```
  <label>Date:</label>
```

```
  <input type="date" formControlName="date">
```

```
</form>
```

Übung 22: FormBuilder

- Bereite die Anwendung für die Verwendung der Forms-API vor
 - Importiere `ReactiveFormsModule` im `imports`-Array von `CustomerDetailsComponent`
- Implementiere in `CustomerDetailsComponent` ein Formular
 - Verwende die typisierte Forms-API (`FormGroup/FormControl` oder `inject(FormBuilder).nonNullable`) und speichere das Formular in einem Feld
 - Erstelle für jedes Feld in `Customer` (außer `ID` und `image`) ein `FormControl`
 - Initialisiere die Felder mit den vom Service geholten Daten
 - Implementiere ein passendes Template mit einem Label und passendem Input pro `FormControl`
 - Tip: `<input type="date">` für `lastOrderDate`, Daten-Mapping erforderlich
`customer.lastOrderDate.toIsoString().substr(0, 10)` und `new Date(inputValue)`
- Implementiere das Speichern der geänderten Daten
 - Verwende das `ngSubmit`-Event am Formular und einen Submit-Button zum Speichern
 - Übernehme die geänderten Daten in das Domain-Objekt (Tip: `Object.assign`)



Validierung

Prüfen der Formulareingaben

- 2-fache Validierung ist nicht redundant
 - Client-seitig Validierung für User Experience
 - Server-seitig Validierung für Sicherheit
- Validatoren
 - Build-in
 - Custom
- Verwendung
 - Mittels Attributierung bzw. Direktive
 - Mit Hilfe der Forms API

Verwendung

```
this.form = new FormGroup({  
  name: new FormControl("", Validators.required),  
  city: new FormControl("", Validators.maxLength(10)),  
  zip: new FormControl("", Validators.pattern('[0-9]{5}')),  
  street: new FormControl("", Validators.compose([  
    Validators.required, Validators.minLength(3)  
  ]))  
})
```

Validierungsfehler

- Zustand pro Feld / Gruppe / Formular
 - untouched, touched
 - pristine, dirty
 - valid, errors
 - Value
- Zusammenspiel mit CSS Klassen
 - Angular setzt CSS-Klassen an Formular und Feldern
 - ng-valid, ng-invalid, ng-pristine, ng-dirty

Zugriff auf Fehlerzustand

```
<div class="form-group"  
[class.has-error]="form.controls.email.dirty &&  
!form.controls.email.valid"  
[class.has-warning]="form.controls.email.pristine &&  
!form.controls.email.valid">  
<input type="email" class="form-control"  
formControlName="email">  
</div>
```

ErrorsMap

```
<div [hidden]="!form.controls.password.dirty">
```

```
  <div [hidden]="!form.controls.password.errors?.required">
```

Password ist Pflichtfeld

```
  </div>
```

```
  <div [hidden]="!form.controls.password.errors?.complex">
```

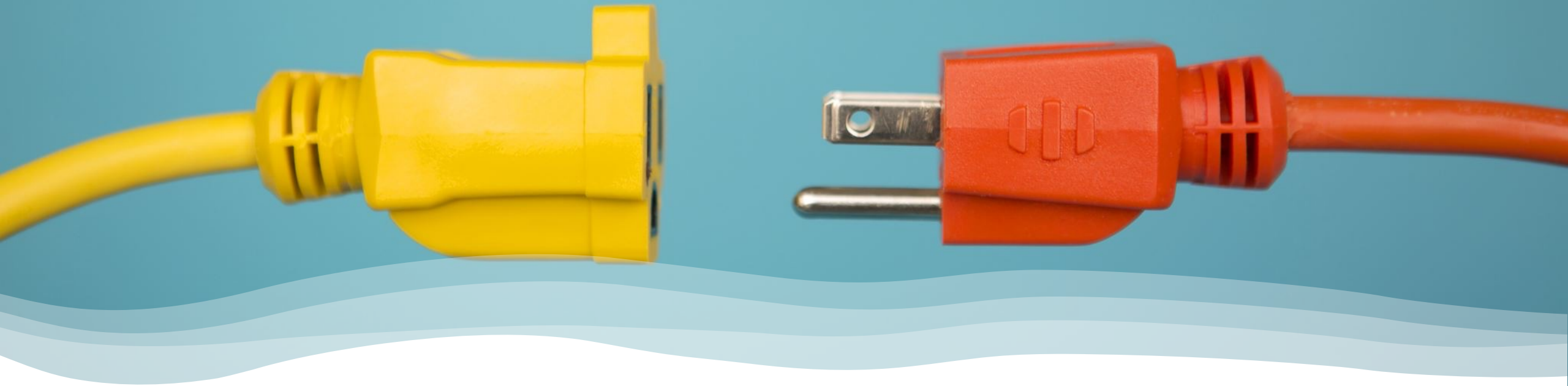
Password muss aus 8 Zeichen und mind. 1 Sonderzeichen und Großbuchstaben bestehen.

```
  </div>
```

```
</div>
```

Übung 22: Validierung

- Validiere in CustomerDetailComponent
 - First- und lastname required
 - Zip als string aber nur zahlen
- Gib für jedes invalide Feld eine Fehlermeldung aus
- Verhindere das Absenden des invaliden Formulars
 - Tip: [disabled] an Submit-Button



Modulare wiederverwendbare Komponenten

Kind Komponenten

Best Practice

- Smarte Komponenten und einfache (dumme) Komponenten
 - Smarte Komponenten wissen etwas über Umgebung
 - z.B. Abhängigkeiten von Services
 - Dumme Komponenten haben nur Inputs und Outputs
 - z.B. keine Abhängigkeiten von Services
- Bessere Wiederverwendbarkeit bei dummen Komponenten
- In unserer Beispielanwendung Formular aus Detailkomponente auslagern. → **Wie läuft die Interaktion topDown und bottomUp**

Rückblick: Input Output

- Vater zu Kind über Property-Binding
 - Daten von oben nach unten übergeben
 - Kind: Signal-Input `input()` – Reaktion auf Änderungen per `computed()` / `effect()` statt `ngOnChanges`
- Kind zu Vater über Event-Binding mit `output()`
 - Events von unten nach oben reichen

viewChild() – Signal Query

- Injection von Kindkomponente zur einfacheren Interaktion

```
export class ParentComponent {  
  listComponent = viewChild(ListComponent); // Signal<ListComponent  
  | undefined>  
  refresh() { this.listComponent()?.reload(); }  
}
```

- Legacy: @ViewChild(ListComponent) private listComponent:
 ListComponent;

Kommunikation über ThirdParty

- Kommunikationskanal
 - Service stellt Observables bereit
- Auch andere Kommunikationsstrukturen möglich z.B. 1:n
- Vaterkomponente kann als Provider vom Service dienen (Scoped Service Instanziierung)

Beispiel Kommunikations-Service

```
@Component({ selector: 'list', ... })
class ListComponent {
  private readonly com = inject(Communication);
  constructor() {
    this.com.detailToList.pipe(takeUntilDestroyed())
      .subscribe((update: DetailUpdate) =>
        console.log("update received"));
  }
  doSomething() {
    this.com.listToDetail.next(new ListUpdate());
  }
}
```

```
@Injectable()
class Communication {
  listToDetail = new Subject<ListUpdate>();
  detailToList = new Subject<DetailUpdate>();
}

@Component({
  imports: [ListComponent, DetailComponent],
  template: `<list /><detail />`,
  providers: [Communication]
})
class MasterDetailComponent {}
```

Übung 23

- Stelle in CustomerService eine Methode update bereit
 - Rufe die Methode beim Speichern des Formulars in CustomerDetailsComponent auf
 - Ersetze in der Methode das ursprüngliche Customer-Objekt durch das neue
 - Feuere einen neuen Datensatz am internen Subject
- Zeige in AppComponent immer den letzt aktiven Customer an
 - Implementiere in CustomerService die Logik (Methode getNext(): Observable<Customer>)
 - Nutze dafür das Observable von getAll
 - Nutze eine flache Kopie des Arrays zum Sortieren (array.slice().sort)
 - Hole in AppComponent das Observable von getNext und wandle es per toSignal() in ein Signal um
 - Zeige im UI den Namen des nächsten Customer an

The image features a solid blue background. Two white telephone receivers are positioned horizontally, one above the other. Each receiver is connected to a coiled white cord that extends towards the left and right edges of the frame. The receivers are slightly angled, with the top one pointing towards the upper right and the bottom one towards the lower right. Overlaid in the center of the image, between the two receivers, is the text "Remote-Kommunikation" in a white, sans-serif font.

Remote-Kommunikation

A photograph of a small, vibrant green plant with several rounded leaves growing out of a crack in a dark, textured asphalt surface. The background is a blurred, light-colored sky with a warm, orange glow in the upper right corner. The text 'Exkurs REST' is overlaid in white, sans-serif font across the middle of the image.

Exkurs REST

Das http-Protokoll

- Eine umfassende Spezifikation des w3w-Konsortiums
 - Siehe <http://en.wikipedia.org/wiki/Http>

Hypertext Transfer Protocol

From Wikipedia, the free encyclopedia
(Redirected from [Http](#))

The **Hypertext Transfer Protocol (HTTP)** is an [application protocol](#) for distributed, collaborative, [hypermedia](#) information systems.^[1] HTTP is the foundation of data communication for the [World Wide Web](#).

[Hypertext](#) is structured text that uses logical links ([hyperlinks](#)) between [nodes](#) containing text. HTTP is the protocol to exchange or transfer hypertext.

The standards development of HTTP was coordinated by the [Internet Engineering Task Force](#) (IETF) and the [World Wide Web Consortium](#) (W3C), culminating in the publication of a series of [Requests for Comments](#) (RFCs), most notably [RFC 2616](#) (June 1999), which defines HTTP/1.1, the version of HTTP in common use.

Contents [\[hide\]](#)

- 1 Technical overview
- 2 History
- 3 HTTP session
- 4 Request methods
 - 4.1 Safe methods
 - 4.2 Idempotent methods and web applications
 - 4.3 Security

Internet protocol suite

Application layer
BGP • DHCP (DHCPv6) • DNS • FTP • **HTTP** • IMAP • IRC • LDAP • MGCP • NNTP • NTP • POP • RPC • RTP • RTSP • RIP • SIP • SMTP • SNMP • SOCKS • SSH • Telnet • TLS/SSL • XMPP • *more...*

Transport layer
TCP • UDP • DCCP • SCTP • RSVP • *more...*

Internet layer
IP (IPv4 • IPv6) • ICMP • ICMPv6 • ECN • IGMP • IPsec • *more...*

Link layer
ARP/InARP • NDP • OSPF • Tunnels (L2TP) • PPP • Media access control (Ethernet) • DSL • ISDN • FDDI • DOCSIS) • *more...*

Elemente der http-Spezifikation

- Definition von URIs
 - Pfad
 - Parameter
- http-Request und http-Response
 - Daten-Container mit Header und Body
 - Encodierung
- Umfassender Satz von Header-Properties
 - Content-Length
 - Accepts
 - Content-Type

Elemente der http-Spezifikation II

- http-Methoden
 - PUT
 - GET
 - POST
 - DELETE
 - OPTIONS
 - HEAD
- Statuscodes für Aufrufe
 - 404: „Not found“
 - 204: „Created“
 - ...

MimeTypes

- Definition der Datentypen des Internet
 - Nicht zu verwechseln mit einem XML-Schema
 - Ein MimeType ist „nur“ eine strukturierte Zeichenkette
 - Eigene Erweiterungen sind möglich

REST und http

- REST hat mit http prinzipiell nichts zu tun
 - REST ist eine abstrakte Architektur
 - http ist ein konkretes Kommunikationsprotokoll
- Aber
 - http passt als Kommunikations-Protokoll der „Referenz-Implementierung“
Internet natürlich perfekt zum REST-Stil

GET

- Liefert die Ressource

DELETE

- Löscht die Ressource

PUT

- Erzeugt aktualisiert die Ressource
- Client spezifiziert resourceIdentifier
- PUT /resources/<newResourceId> HTTP/1.1

POST

- Erzeugt ein neues Objekt unterhalb (Collection) der spezifizierten Ressource
- POST /resources HTTP/1.1

Der http Client

Der Service HttpClient

- HttpClient ist ein Service
 - und steht damit via Dependency Injection zur Verfügung: `inject(HttpClient)`
- `provideHttpClient()` in `main.ts` registrieren
 - optional `withFetch()`, `withInterceptors([...])`
- Das API
 - umfasst die Standard-http-Methoden
 - ist asynchron konzipiert (Observables)

```
this.http.get<Data>('/endpoint').subscribe(data => {  
  this.results = data.jsonAttribute;  
});
```

Json Daten extrahieren

```
private readonly http = inject(HttpClient);  
getAll(): Observable<Release[]> {  
    return this.http.get<{ data: Release[] }>('api/release').pipe(  
        map(response => response.data),  
        catchError(this.handleError)  
    );  
}  
// JSON-Parsing übernimmt HttpClient automatisch
```

Liefert Observable

Exception Handler

```
private handleError(error: HttpResponse) {  
    let message;  
    if (error.message) {  
        message = error.message;  
    }  
    else if (error.status) {  
        message = '${error.status} - ${error.statusText}';  
    }  
    else {  
        message = 'Server error';  
    }  
  
    // Rethrow in intermediate Methods  
    return throwError(() => new Error(message));  
}
```

Header

```
const headers = new HttpHeaders({ 'Content-Type': 'application/json'
});
return this.http.post<{ data: MyObject }>('api/objects', object, {
  headers }).pipe(
  map(response => response.data),
  catchError(this.handleError)
);
// JSON.stringify übernimmt HttpClient automatisch
```



Übung 24: Starten des Servers

- Zip-Datei *server-js.zip* entpacken
- Konsole öffnen und in Server Verzeichnis wechseln
- Installieren per npm install
- Starten über node server.js
- Backend läuft unter localhost:3000
- dev.proxy.json: { "/api": { "target": "http://localhost:3000" } }
- Dev-Server mit Proxy starten ([CORS](#)) ng serve --proxy dev.proxy.json

Übung 24: Endpunkte

- GET
 - api/customer (GET_ALL)
 - api/customer/:id (GET_BY_ID)
- POST api/customer (INSERT)
- PUT/PATCH/DELETE api/customer/:id (UPDATE, PARTIAL_UPDATE, REMOVE)

Übung 24: http

- Verwende in CustomerService das Backend (inject(HttpClient)) statt der Mock-Daten
- Registriere provideHttpClient() in main.ts
 - Dev-Server mit Proxy starten
 - Lade die Liste initial vom Backend
 - Achtung: Date Felder vom Server werden serialisiert als String gesendet
 - Müssen noch in Date umgewandelt werden
 - Schicke die geänderten Daten in update per put zum Backend
 - Lade (der Einfachheit halber) nach einem update die Liste neu vom Backend
 - Musterlösung: 23_communication (enthält bereits die HTTP-Anbindung)

Ausblick: Resource API

```
export class CustomerDetailsComponent {  
  private readonly customerService = inject(CustomerService);  
  id = input.required<string>();  
  // lädt automatisch neu, sobald sich id() ändert  
  customer = httpResource<Customer>(() => `api/customer/${this.id()}`);  
  // Alternative mit bestehendem Observable-Service  
  customer2 = rxResource({  
    params: () => this.id(),  
    stream: ({ params }) => this.customerService.getById(params)  
  });  
}  
  
<!-- Template -->  
@if (customer.isLoading()) { <p>Lade...</p> }  
@if (customer.hasValue()) { <p>{{ customer.value().firstname }}</p> }  
// Resource APIs sind teilweise noch experimentell
```

Eigene Direktiven

- Können simpel erstellt werden
- `ng generate directive <directive-name>`
- Die Direktive ist wiederum eine annotierte TypeScript-Klasse
 - Der `selector` ist das neue HTML-Attribute oder -Element
 - Das DOM-Element, das manipuliert werden soll, wird per `inject(ElementRef)` bereitgestellt

Eigene Directive: Beispiel

- Ein Beispiel ist unter <https://angular.dev/guide/directives/attribute-directives> zu finden
- `import { Directive, ElementRef, inject } from '@angular/core';`
- `@Directive({`
- `selector: '[appHighlight]'`
- `})`
- `export class HighlightDirective {`
- `private readonly el = inject(ElementRef);`
- `constructor() {`
- `this.el.nativeElement.style.backgroundColor = 'yellow';`
- `}`
- `}`
- Verwendung: HighlightDirective im imports-Array der Komponente

Legacy: Angular Module (NgModule)

Was ist ein Modul

- Angular-spezifischer Code muss organisiert werden
 - Gleiche Grundgedanken und Aufbau wie bei ES6 Modulen
 - Kapselung & Wiederverwendbarkeit
 - Leichtes Einbinden von Bibliotheken
 - Genaue Steuerung was wo verwendet wird
 - Kompromiss zwischen
 - "alles immer angeben müssen" (Angular < 2.0.0-rc.5) und
 - "einfach rein schmeißen" (AngularJS 1)

Übersicht der Dateien eines Moduls

- *src/index.html*
 - Die Hauptseite
- *src/app/app.module.ts*
 - Definition des Moduls
- *src/app/app.component.ts*
 - Definition des dynamischen Anteils einer Komponente mit TypeScript
- *src/app/app.component.html*
 - Definition des HTML-Templates einer Komponente
- *src/app/app.component.css*
 - Stylesheet der Komponente
- *src/app/app.component.spec.ts*
 - Definition von Karma-Tests

Referenz auf Module in Index-Seite

```
index.html x
1  <!doctype html>
2  <html lang="en">
3  <head>
4      <meta charset="utf-8">
5      <title>org.Javacream.Training.Angular</title>
6      <base href="/">
7
8      <meta name="viewport" content="width=device-width, initial-scale=1">
9      <link rel="icon" type="image/x-icon" href="favicon.ico">
10 </head>
11 <body>
12     <app-root></app-root>
13 </body>
14 </html>
15
```

Der
Einstiegspunkt
für Angular

Component-Annotation

- Eine Component ist eine TypeScript-Klasse mit der `@Component`-Annotation
 - Annotationen enthalten Metadaten, die der Klasse zugeordnet werden
 - Diese müssen von einem Framework interpretiert werden
- Annotationen sind damit so etwas wie statische Eigenschaften einer Klasse
 - Allerdings sind diese syntaktisch nicht Bestandteil der Klassendeklaration
 - Damit ergibt sich eine saubere Trennung
- Eine typische Component definiert
 - Den Selector als Bezug zum Template
 - Die Lokation des Templates
- Vollständige Liste unter
 - *<https://angular.io/api/core/Component>*

Aufgabe der Component

- Die Eigenschaften der Component-Klasse dienen als Schnittstelle zum HTML-Code
 - Werte werden über Interpolation oder Direktiven gebunden
 - Funktionen dienen als Event-Handler
- Bei Bedarf implementiert die Component-Klasse Lifecycle-Funktionen
 - `ngOnChanges`
 - `ngOnInit`
 - *<https://angular.io/guide/lifecycle-hooks>*

Angular-Module: Die Component

```
import {ApplicationRef, Component, OnDestroy, OnInit} from "@angular/core";  
  
@Component({  
  selector: "app-root",  
  templateUrl: "./app.component.html",  
  styleUrls: ["./app.component.scss"],  
  animations: [fadeAnimation]  
})  
export class AppComponent implements OnInit, OnDestroy {  
  
  public title = "theFrontend";  
}
```

TypeScript-Import der
Angular-Core-Bibliothek

Referenz auf den
Einstiegspunkt

Referenz auf Template
und CSS

Definition einer
Variablen und
Wert-Zuweisung

Angular-Module: Das HTML-Template

```
app.component.html x
1  <!--The content below is only a placeholder and can be replaced.-->
2  <div style="text-align:center">
3    <h1>
4      Welcome to the great {{ title }}!
5    </h1>
6    
9  </div>
10 <h2>Here are some links to help you start: </h2>
11 <ul>
12   <li>
13     <h2><a target="_blank" rel="noopener" href="https://angular.io/t
14   </li>
15   <li>
16     <h2><a target="_blank" rel="noopener" href="https://github.com/a
17   </li>
18   <li>
19     <h2><a target="_blank" rel="noopener" href="https://blog.angular
```

Referenzieren der
Component-Variable mit
Interpolation

Angular-Module: Module-Deklaration

```
TS app.module.ts x
1  import { BrowserModule } from '@angular/platform-browser';
2  import { NgModule } from '@angular/core';
3
4
5  import { AppComponent } from './app.component';
6
7
8  @NgModule({
9    declarations: [
10     AppComponent
11   ],
12   imports: [
13     BrowserModule
14   ],
15   providers: [],
16   bootstrap: [AppComponent]
17 })
18 export class AppModule { }
19 |
```

Import von Angular-Core
und dem Browser-
Support

Import der eigenen
Komponente

Definition aller
Eigenschaften des
Moduls

Durch den Export wird
das Modul aktiviert

Ablauf bis hier her

- Ablauf bis hier
 - Aufruf bootstrapModule mit Modul (AppModule)
 - Modul ist Referenz auf eine Klasse
 - Klasse hat einen Decorator
 - Modul-Decorator enthält Komponenten
- Woher weiß Angular welche Komponente für das Bootstrapping verwendet werden soll?

Angular-Module: Module-Deklaration

```
TS app.module.ts x
1  import { BrowserModule } from '@angular/platform-browser';
2  import { NgModule } from '@angular/core';
3
4
5  import { AppComponent } from './app.component';
6
7
8  @NgModule({
9    declarations: [
10     AppComponent
11   ],
12   imports: [
13     BrowserModule
14   ],
15   providers: [],
16   bootstrap: [AppComponent]
17 })
18 export class AppModule { }
19 |
```

„Main“ Component

Kompletter Ablauf

- Ablauf insgesamt
 - Aufruf bootstrapModule mit Modul (AppModule)
 - Modul ist Referenz auf eine Klasse
 - Klasse hat einen Decorator
 - Modul-Decorator hat Feld bootstrap
 - Feld bootstrap hält Referenz auf Komponenten Klasse (AppComponent)
 - Komponenten-Klasse hat Decorator
 - Komponenten-Decorator hat Feld selector
 - Selector wird in DOM aus index.html gesucht
 - Komponente wird instanziiert

Deklaration, Imports und Exports

```
@NgModule({  
  declarations: [ ReleaseListComponent, ReleaseCardComponent ],  
  imports: [ BrowserModule ],  
  exports: [ ReleaseListComponent ]  
})  
export class ReleaseModule { }
```

Module generieren

- Angular CLI legt neue Module an
 - Module-Klasse + Annotation
 - Module-Komponente
- `ng generate module name`
- Wird *NICHT* automatisch am nächsthöheren gelegenen Modul registriert → selbst machen

Legacy: DI Konfig

- Woher weiß Angular was zu injizieren ist?
 - Muss konfiguriert werden: *Providers*
 - Injector -> Provider -> Instance
- Konfiguration an Modulen

```
@NgModule({  
  providers: [ ] /* array of providers */  
})  
  
export class TrackModule { }
```
- Wie sieht ein Provider aus?

Legacy: Wie geht DI in Angular

- Direkte Abhängigkeit
 - einfach nur die Klasse angeben: [ConsoleLogger]
 - Kurz für [{ provide: ConsoleLogger, useClass: ConsoleLogger}]
- Allgemein: Provider-Objekt
 - provide: Unter welchem Namen steht es zur Verfügung -> Klasse
 - Ein Rezept, wie soll es erzeugt werden soll -> useClass, useValue, useFactory oder useExisting

```
@NgModule({  
  providers: [  
    { provide: ConsoleLogger, useClass: ConsoleLogger }, // same as shortcut  
    { provide: Logger, useExisting: ConsoleLogger } // different token, same instance  
  ]  
})  
export class TrackModule { }
```

Legacy: Deklaration an Modulen

```
@NgModule({  
  declarations: [ HasLikesPipe ],  
  exports: [ HasLikesPipe ]  
})  
export class AppModule { }
```

Legacy: Routenkonfiguration

- Wird verwendet um Provider für Router Service zu erzeugen
- Provider wird von RouterModule bereitgestellt
 - RouterModule muss in AppModule importiert werden
 - forRoot darf nur einmal im AppModule verwendet werden
 - forChild nur bei Lazy-Loading

Legacy: Beispiel: Routen-Definition

```
const appRoutes: Routes = [  
  { path: 'path1', component: Component1},  
  { path: ' ', redirectTo: '/index', pathMatch: 'full'},  
  { path: '**', component: PageNotFoundComponent }  
];  
  
@NgModule({  
  imports: [  
    RouterModule.forRoot(  
      appRoutes,  
      { enableTracing: true } // <-- debugging purposes only  
    )  
  ],  
  ...  
})  
  
export class AppModule { }
```

Legacy: Location-Strategy

- LocationStrategy festlegen
 - PathLocationStrategy (**default**)
 - Bessere SEO Indizierung
 - HashLocationStrategy: *default AngularJS 1*
 - Simpler
- Verwendung von **PathLocationStrategy**
 - <base href=". "> als erstes Element im head der index.html

```
import { RouterModule } from '@angular/router';  
import { routes } from './app/app.routes';
```

```
@NgModule({ imports: [  
  RouterModule.forRoot(routes, { useHash: true })  
] })  
export class AppModule { }
```

Legacy: ActivatedRoute und params.subscribe

```
import { ActivatedRoute } from "@angular/router";
export class ArtistDetailsComponent implements OnInit, OnDestroy {
  private subscription: Subscription;

  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    this.subscription = this.route.params
      .subscribe(params => console.log(params['id']));
  }

  ngOnDestroy() { this.subscription.unsubscribe(); }
}
```

Zugriff auf Pfadparameter

Legacy: Der Service HttpClient

- HttpClient ist ein Service
 - und steht damit via Dependency Injection zur Verfügung
 - **HttpModule in AppModule importieren**
 - XMLHttpRequest führt Requests aus
- Das API
 - umfasst die Standard-http-Methoden
 - ist asynchron konzipiert

```
http.get('/endpoint').subscribe(data => {  
    // Read the result field from the JSON response.  
    this.results = data['jsonAttribute'];  
});
```

Übung 25: Legacy – Angular Module

- Öffne das Projekt 25_modules_legacy (NgModule-basierte Variante)
 - Backend starten (server: npm install, node server.js)
 - App starten mit ng serve --proxy-config dev.proxy.json
- Analysiere den Aufbau
 - main.ts: platformBrowserDynamic().bootstrapModule(AppModule)
 - AppModule: declarations, imports, providers, bootstrap
 - TrainingsModule und AppRoutingModuleModule (RouterModule.forRoot)
- Erweitere das Modul
 - Erzeuge eine Komponente per ng g c trainings/training-info --standalone=false und deklariere sie im TrainingsModule
 - Binde eine Standalone-Komponente über imports: [] in ein NgModule ein (Mischbetrieb)
- Optional: Migriere das Projekt
 - ng generate @angular/core:standalone ausführen und mit 23_communication vergleichen

Anhang



node.js

Was ist node.js?

- node.js ist ein Interpreter für Server-seitiges JavaScript
 - Auf Grundlagen der Google V8-Engine
- Mit node.js können damit keine Browser-Anwendungen betrieben werden
 - Keine UI, Keine User-Events
 - Kein Html-Dokument und damit kein DOM
 - Kein Browser-API
 - Window
 - Historie
 - ...
- Dafür stellt node.js eigene Bibliotheken zur Verfügung
 - Dateizugriff
 - Multithreading
 - Networking
 - ...
 - <https://nodejs.org/dist/latest-v8.x/docs/api/>

Beispiel: Ein kompletter http-Server

```
var http = require('http');
var fs = require('fs');
http.createServer(function handler(req, res) {
  var url = req.url;
  if (url.match(/.html/)) {
    res.writeHead(200, {
      'Content-Type' : 'text/html'
    });
  } else if ...
  var filename = "./static-content" + req.url;
  fs.createReadStream(filename).pipe(res);
}).listen(6061, '127.0.0.1');
```

Installation: node.js

LTS
Recommended For Most Users

Current
Latest Features


Windows Installer
node-v6.11.4-x86.msi


Macintosh Installer
node-v6.11.4.pkg


Source Code
node-v6.11.4.tar.gz

Windows Installer (.msi)
Windows Binary (.zip)
macOS Installer (.pkg)
macOS Binaries (.tar.gz)
Linux Binaries (x86/x64)
Linux Binaries (ARM)
Source Code

32-bit	64-bit	
32-bit	64-bit	
64-bit		
64-bit		
32-bit	64-bit	
ARMv6	ARMv7	ARMv8
node-v6.11.4.tar.gz		

Additional Platforms

SunOS Binaries
Docker Image
Linux on Power Systems
Linux on System z
AIX on Power Systems

32-bit	64-bit
Official Node.js Docker Image	
64-bit le	64-bit be
64-bit	
64-bit	

Testen der Installation

```
node -v
```

- Ausgabe der Versionsnummer

```
node
```

- Starten der REPL zur Eingabe von JavaScript-Befehlen

```
node programm.js
```

- Ausführen der Skript-Datei *programm.js*

Node und Browser-basierte Anwendungen

- Obwohl node.js nicht im Browser ausgeführt wird, wird es trotzdem gerne im Rahmen der Software-Entwicklung genutzt
- Hierzu wird node als Web Server eingesetzt, der die JavaScript-Dateien sowie die statischen Ressourcen (HTML, CSS, ...) zum Browser sendet
 - Mit Hilfe eines Browser-Sync-Frameworks triggern Änderungen von JavaScript-Dateien auf Server-Seite einen Browser-Refresh
 - <https://www.browsersync.io/>
 - Damit werden Änderungen ohne weitere Benutzer-Interaktion sofort angezeigt
 - Für eine agile Software-Entwicklung natürlich äußerst praktisch

npm – Der Node Package Manager



Was ist npm?

- Primär ein Packaging Manager
- `npm` ist Bestandteil der `node`-Installation
 - `npm -v`
- Die offizielle npm Registry liegt im Internet
 - <https://docs.npmjs.com/misc/registry>
 - Im Wesentlichen eine CouchDB
 - Laden der Software durch RESTful Aufrufe
 - Die `npm`-Registry ist aktuell die größte Sammlung von Software
- Unternehmens-interne oder private Registries können angemietet werden

npm Kommandos

- npm wird über die Kommandozeile angesprochen
 - eine grafische Oberfläche wird als separates Modul zur Verfügung gestellt
- Hilfesystem
 - `npm -h`
 - `npm <command> -h`
 - <https://docs.npmjs.com/>



Node-Modules

Node Modules

- Jede via `npm` geladene Bibliothek wird als Node-Module konzipiert
- Jedes Modul besitzt
 - Eine Informationsdatei, die `package.json`, die das Projekt zusätzlich beschreibt
 - Abhängige Bibliotheken im Unterverzeichnis `node_modules`
 - Diese sind selbst ebenfalls Node-Module
 - Einen Entry-Point, in dem der Module-Entwickler das Fachobjekt seines Moduls erzeugt und exportiert
 - Dazu wird dem `module`-Objekt die Eigenschaft `exports` gesetzt
 - Zur Benutzung eines Moduls innerhalb eines Scripts dient der Node-Befehl `require`
 - Der Rückgabewert von `require` ist das vom Modul erzeugte und exportierte Fachobjekt

Die package.json

- Enthält die Projektinformation im JSON-Format
- Die Datei enthält
 - Den Projektnamen
 - Die aktuelle Versionsnummer
 - Meta-Informationen wie Autor, Schlüsselwörter, Lizenz
 - Dependencies
 - Ein `scripts`-Objekt mit ausführbaren Befehlen
 - Diese können mit `npm run <script>` ausgeführt werden

Initialisierung eines Projekts

- Jedes `npm`-basierte Projekt ist ein neues Node-Module
- Initialisierung mit `npm init`
 - Dabei werden interaktiv die Informationen abgefragt, die zur Erstellung der initialen `package.json` benötigt werden

Projektstruktur

project-root

package.json

src

- script1
- script2
- ...

node-modules

- module1
- module2
- ...

Beispiel: Ein einfaches Projekt

```
{  
  "name": "npm-sample",  
  "version": "1.0.0",  
  "description": "a simple training project",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [  
    "training"  
  ],  
  "author": "theAuthor",  
  "license": "ISC"  
}
```

Beispiel: Ein einfaches Node-Module

- Datei *index.js*

```
module.exports = {  
  log: function() {  
    console.log('Hello')  
  }  
}
```

- In der REPL

```
var training = require('./index.js')  
training.log()
```

Installieren von Abhängigkeiten

- Abhängigkeiten werden mit `npm install` von einer npm-Registry geladen
 - Ohne weitere Konfiguration wird dazu die Standard-Registry benutzt
 - Damit ist eine Internet-Verbindung notwendig
 - Es können aber auch Unternehmens-interne Repository-Server benutzt werden
 - z.B. Nexus
- Rechner-Registry
 - Die Abhängigkeiten werden auf dem Rechner abgelegt
 - Ab jetzt ist damit keine Internet-Verbindung mehr nötig
 - Orte:
 - lokale Ablage in einem Unterverzeichnis namens *node-modules*
 - Empfohlenes Standard-Verfahren zur Installation von Dependencies für eigene Software-Projekte
 - globale Ablage
 - Empfohlenes Standard-Verfahren zur Installation von allgemein verwendbaren Werkzeugen